

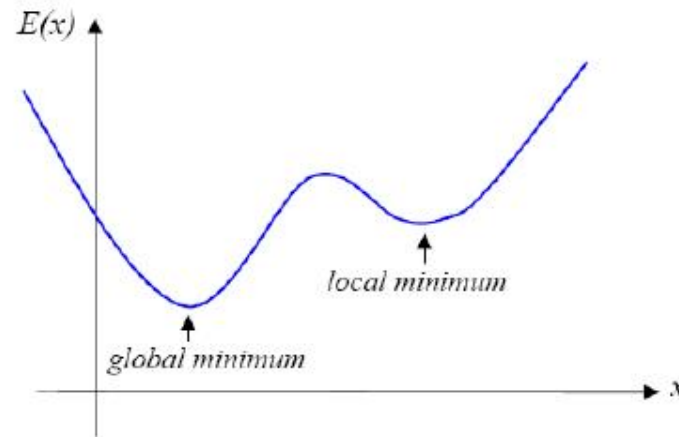
Neural Networks

EEE 443/543

Lecture 6-
Optimization &
Generalization

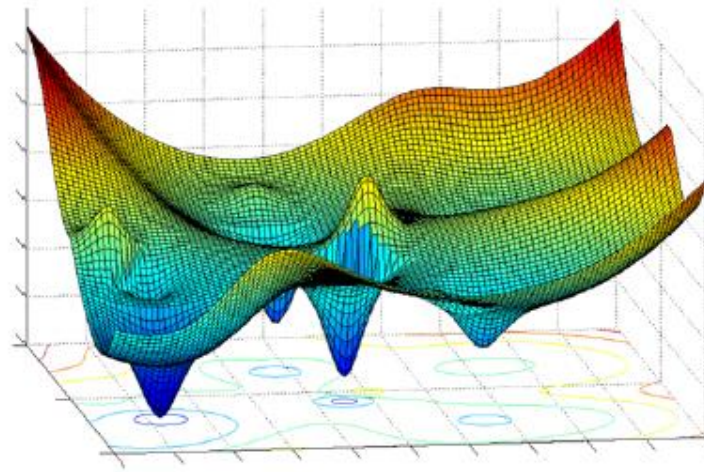
Local Minima

Cost functions can quite easily have more than one minimum:



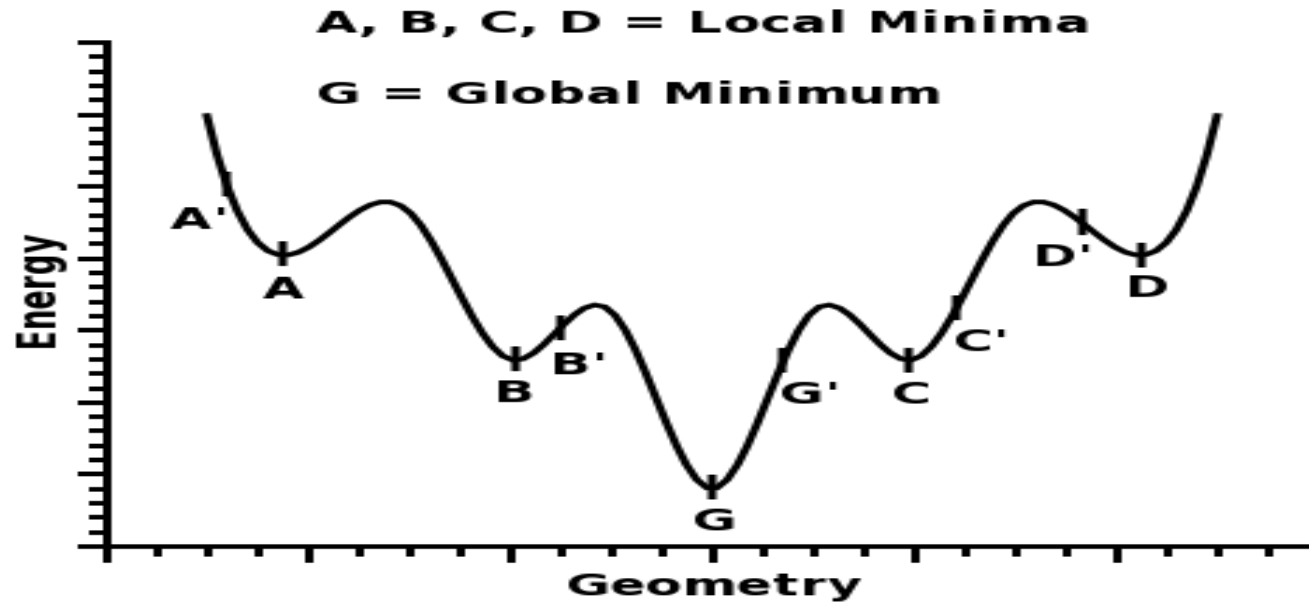
If we start off in the vicinity of the local minimum, we may end up at the local minimum rather than the global minimum.

When training very deep networks, we are optimizing a **non-convex** function over **millions** of parameters.



Lots of saddle points and local minima!

- LOCAL MINIMUM PROBLEM...



- Convergence IS NOT GUARANTEED ...
- “ If η is sufficiently small and if $W(0)$ is sufficiently close to a local minimum then convergence to that local minimum is guaranteed”....

- **Stochastic Gradient Descent :**

- In most of the cases, same as SHUFFLING

$$\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$$

$$I = \{ \overset{\uparrow}{1} \overset{\uparrow}{2} \dots \overset{\uparrow}{N} \}$$

Update order

→ $sh(I) = \{ sh(1) sh(2) \dots sh(N) \}$

New update order

$$sh(i) = random(1, N) \neq sh(j) \quad j \leq i$$

Same as RANDOM PERMUTATION

- With this update order, use incremental weight update
- At the end, start a new epoch

- **Stochastic Gradient Descent :**

- In most of the cases, same as SHUFFLING

$$\mathcal{S}_{Tr} = \{x^1, x^2, \dots, x^N\}$$

$$I = \{ \overset{\uparrow}{1} \overset{\uparrow}{2} \dots \overset{\uparrow}{N} \}$$

Update order

→ $sh(I) = \{ sh(1) sh(2) \dots sh(N) \}$

New update order

$$sh(i) = random(1, N) \neq sh(j) \quad j \leq i$$

Same as RANDOM PERMUTATION

- With this update order, use incremental weight update
- At the end, start a new epoch

- Adding noise :

$$W \leftarrow W - \eta \frac{\partial E}{\partial W} + n$$

Small random term (noise)

Choosing the Learning Rate

1. If η is too small, it will take too long to get anywhere near the minimum of the error function.
2. If η is too large, the weight updates will over-shoot the error minimum and the weights will oscillate, or even diverge.

Unfortunately, the optimal value is network dependent.

Generally, one should try a range of different values (e.g. $\eta = 0.1, 0.01, 1.0, 0.0001$) and use the results as a guide.

Batch Training vs. On-line Training

Batch Training: update the weights after all training patterns have been presented.

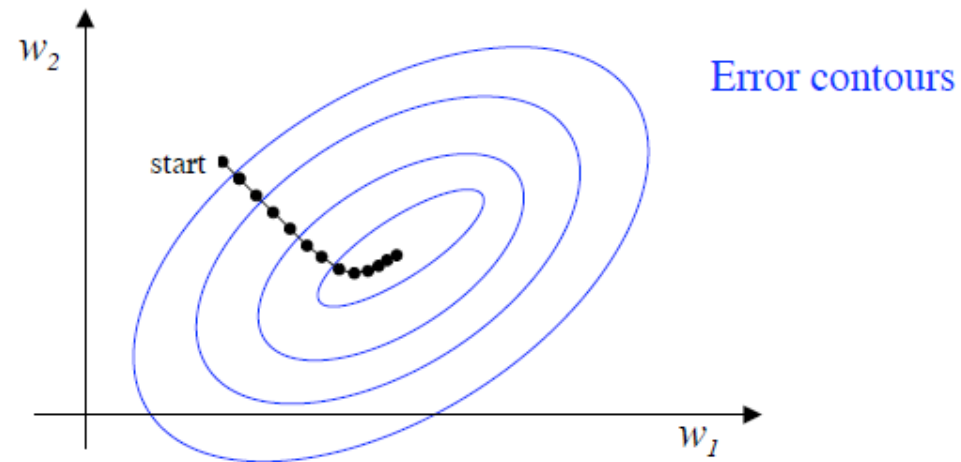
On-line Training (or Sequential Training): A natural alternative is to update all the weights immediately after processing each training pattern.

On-line learning does not perform true gradient descent, and the individual weight changes can be rather erratic. Normally a much lower learning rate η will be necessary than for batch learning.

However, because each weight now has N updates (where N is the number of patterns) per epoch, rather than just one, overall the learning is often much quicker.

Visualising Learning

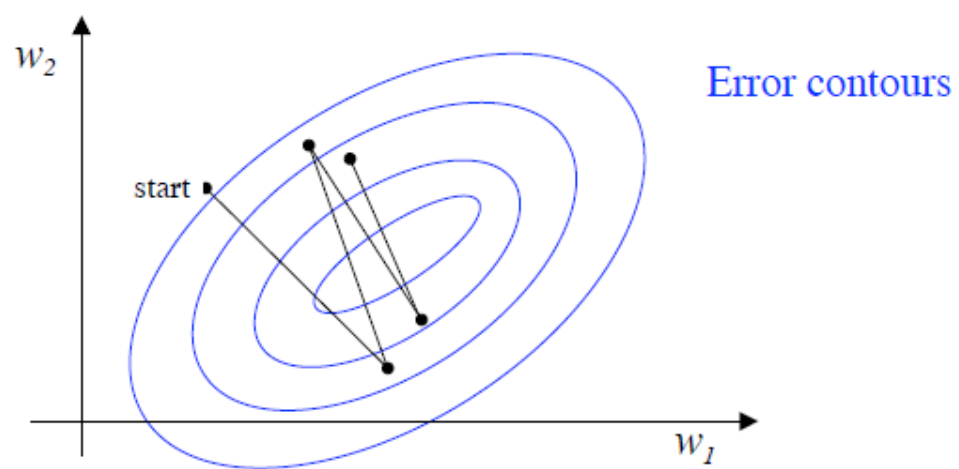
The weight update equations will produce a series of steps in weight space from the starting position to an error minimum:



True gradient descent produces a smooth curve perpendicular to the contours. Weight updates with a small step size η will result in a good approximation to it as shown.

Step Size Too Large

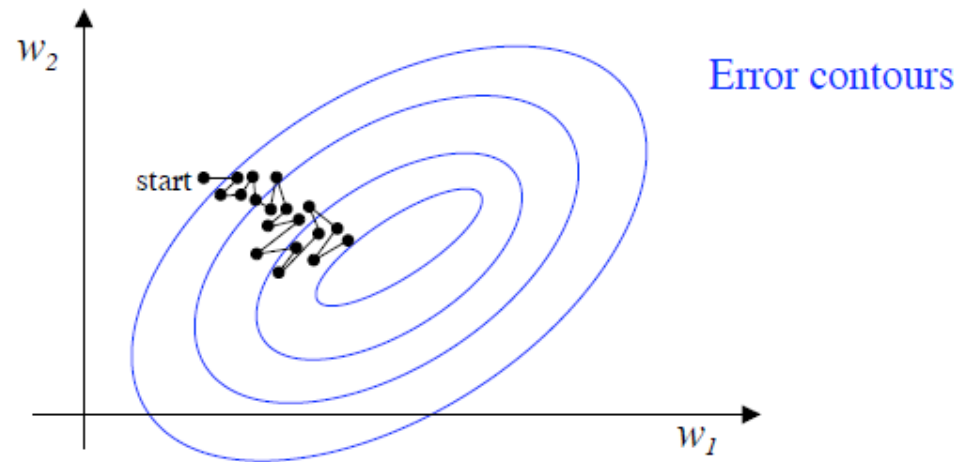
If the step size (i.e. learning rate η) is set too large, the approximation to true gradient descent will be poor, and this will result in overshoots, or even divergence:



If the error function is fluctuating, it would be wise to reduce the step size

On-line Learning

The learning algorithm is no longer true gradient descent, and the weight changes will not be perpendicular to the contours:



If the step sizes are kept small enough, the erratic behaviour of the weight updates will not be too much of a problem, and the increased number of weight changes will still get us to the minimum quicker than true gradient descent (i.e. batch learning).

Question 1 : Setting learning rates

Suppose we have the following loss function over a real number w .

$$C(w) = \frac{1}{2}w^2$$

we would like to minimize this function using gradient descent. so we start at an initial value $w^{(0)} = 1$ and update the weight at time step t as

$$w^{(t+1)} = w^{(t)} - \epsilon \left. \frac{\partial C}{\partial w} \right|_t$$

Compute the weights after the first three steps : $w^{(1)}$, $w^{(2)}$ and $w^{(3)}$ in each of the following situations

- $\epsilon = 0.5$.
- $\epsilon = 1$.
- $\epsilon = 2$.
- $\epsilon = 3$.

in which case(s) do you think, the model will converge if we keep updating w for a long time ?

Question 1 : Setting learning rates

Suppose we have the following loss function over a real number w .

$$C(w) = \frac{1}{2}w^2$$

We would like to minimize this function using gradient descent. So we start with an initial value $w^{(0)}$ and update the weight at time step t as

$$w^{(t+1)} = w^{(t)} - \epsilon \left. \frac{\partial C}{\partial w} \right|_t$$

Compute the weights after the first three steps : $w^{(1)}$, $w^{(2)}$ and $w^{(3)}$ in each of the following situations

- $\epsilon = 0.5$. Converges gradually.
- $\epsilon = 1$. Converges in one step.
- $\epsilon = 2$. Oscillates.
- $\epsilon = 3$. Explodes!

In which case(s) do you think, the model will converge if we keep updating w for a long time ?

MOMENTUM METHOD

$$\Delta W(n) = -\eta \frac{\partial E}{\partial W} + \alpha \Delta W(n-1)$$

Gradient Descent

Momentum term

MOMENTUM METHOD

$$\Delta W(n) = -\eta \frac{\partial E}{\partial W} + \alpha \Delta W(n-1)$$

 Gradient Descent  Momentum term

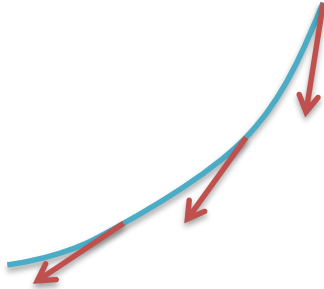
$$\Delta W(1) = -\eta \frac{\partial E(1)}{\partial W}$$

$$\Delta W(2) = -\eta \frac{\partial E(2)}{\partial W} + \alpha \Delta W(1) = -\eta \frac{\partial E(2)}{\partial W} - \eta \alpha \frac{\partial E(1)}{\partial W}$$

$\vdots$

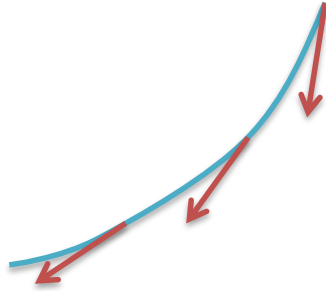
$$\Delta W(n) = -\eta \sum_{k=1}^n \alpha^{n-k} \frac{\partial E(k)}{\partial W}$$

$$\Delta W(n) = -\eta \sum_{k=1}^n \alpha^{n-k} \frac{\partial E(k)}{\partial W}$$

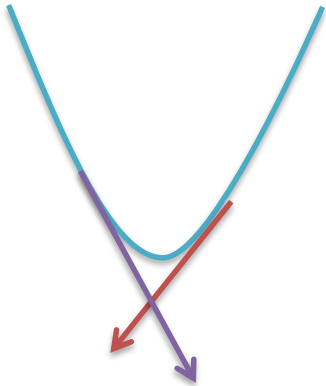


- **Going DOWNHILL :**
- ΔW increases : **faster convergence**

$$\Delta W(n) = -\eta \sum_{k=1}^n \alpha^{n-k} \frac{\partial E(k)}{\partial W}$$



- **Going DOWNHILL :**
- ΔW increases : **faster convergence**



- **Around local minima :**
- ΔW decreases : **eliminates oscillation**

Filter Analogy

- Take Discrete (z-) Transform :

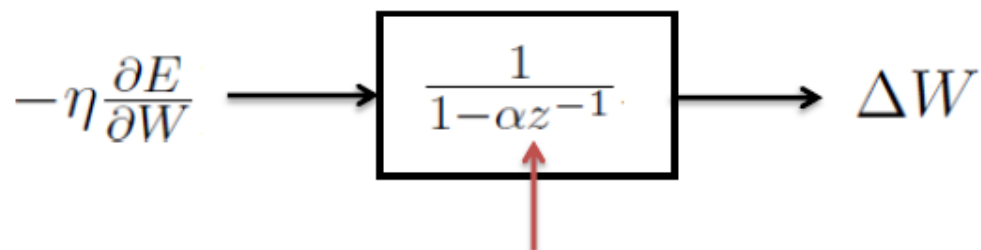
$$\Delta W(n) = -\eta \frac{\partial E}{\partial W} + \alpha \Delta W(n-1)$$

$$\Delta W(z) = \alpha z^{-1} \Delta W(z) + \mathcal{Z}\left(-\eta \frac{\partial E}{\partial W}\right)$$

Filter Analogy

$$\Delta W(z) = \alpha z^{-1} \Delta W(z) + \mathcal{Z}\left(-\eta \frac{\partial E}{\partial W}\right)$$

$$\Delta W(z) = \frac{1}{1 - \alpha z^{-1}} \mathcal{Z}\left(-\eta \frac{\partial E}{\partial W}\right)$$



- Acts as a **LOW PASS FILTER**
- Eliminates **HIGH FREQUENCY OSCILLATIONS**
- **STABLE** for $0 < \alpha < 1$

Question 2 : Momentum

Suppose we have the following loss function over a real number w

$$C(w) = \frac{1}{2} \frac{w^2}{100}.$$

We would like to minimize this function using gradient descent **with momentum**. so we start at an initial value $w^{(0)} = 1, v^{(0)} = 0$ and update the weight at time step t as

$$\begin{aligned} v^{(t+1)} &= \mu v^{(t)} - \epsilon \left. \frac{\partial C}{\partial w} \right|_t \\ w^{(t+1)} &= w^{(t)} + v^{(t+1)} \end{aligned}$$

Compute the weights after the first three steps : $w^{(1)}, w^{(2)}$ and $w^{(3)}$ in each of the following situations

- $\epsilon = 1, \mu = 0$.
- $\epsilon = 1, \mu = 1$.

Which case converges faster?

Question 2 : Momentum

Solution

$$\begin{aligned}\left. \frac{\partial C}{\partial w} \right|_t &= \frac{w^{(t)}}{100} \\ v^{(t+1)} &= \mu v^{(t)} - \epsilon \left. \frac{\partial C}{\partial w} \right|_t \\ w^{(t+1)} &= w^{(t)} + v^{(t+1)}\end{aligned}$$

- $w^{(0)} = 1, \epsilon = 1, \mu = 0.$

No momentum. Therefore $v^{(t+1)} = -\epsilon \left. \frac{\partial C}{\partial w} \right|_t$.

$$w^{(1)} = w^{(0)} - \epsilon \frac{w^{(0)}}{100} = 1 - 1 \cdot \frac{1}{100} = 0.99$$

$$w^{(2)} = w^{(1)} - \epsilon \frac{w^{(1)}}{100} \approx 0.99 - 1 \cdot \frac{0.99}{100} \approx 0.98$$

$$w^{(3)} = w^{(2)} - \epsilon \frac{w^{(2)}}{100} \approx 0.98 - 1 \cdot \frac{0.98}{100} \approx 0.97$$

- $w^{(0)} = 1, \epsilon = 1, \mu = 1.$

$$v^{(1)} = \mu v^{(0)} - \epsilon \frac{w^{(0)}}{100} = 0 - 1 \cdot \frac{1}{100} = -0.01, w^{(1)} = w^{(0)} + v^{(1)} = 1 - 0.01 = 0.99$$

$$v^{(2)} = \mu v^{(1)} - \epsilon \frac{w^{(1)}}{100} = -0.01 - 1 \cdot \frac{0.99}{100} \approx -0.02, w^{(2)} = w^{(1)} + v^{(2)} \approx 0.99 - 0.02 = 0.97$$

$$v^{(3)} = \mu v^{(2)} - \epsilon \frac{w^{(2)}}{100} = -0.02 - 1 \cdot \frac{0.97}{100} \approx -0.03, w^{(3)} = w^{(2)} + v^{(3)} \approx 0.97 - 0.03 = 0.94$$

w goes **faster** towards 0 with momentum.

We saw how momentum works in simple 1-D error functions.

In reality, we'd never use a momentum of 1.

- The weights will oscillate, rather than converge.
- The updates may even blow up.
- However, typical values are very close to 1, e.g. 0.9 to 0.999.

Different Learning Rates for Different Layers?

1. The later network layers (nearer the outputs) will tend to have larger local gradients (*deltas*) than the earlier layers (nearer the inputs).
2. The activations of units with many connections feeding into or out of them tend to change faster than units with fewer connections.
3. Activations required for linear units will be different for Sigmoidal units.
4. There is empirical evidence that it helps to have different learning rates η for the thresholds/biases compared with the real connection weights.

In practice, it is often quicker to just use the same rates η for all the weights and thresholds, rather than spending time trying to work out appropriate differences.

Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution

$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

%80

$$d_1^{train}, \dots, d_N^{train}$$

$$d_1^{test}, \dots, d_M^{test}$$

%20

$$o = F(W, x) \quad \leftarrow \text{Network output}$$

cost (loss) function to be minimized

$$E_{train}(W) = \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W, x_i^{train})\|^2$$

$$W^* = \operatorname{argmin} E_{train}(W)$$

$$E_{train} = E_{train}(W^*)$$

$$= \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W^*, x_i^{train})\|^2$$

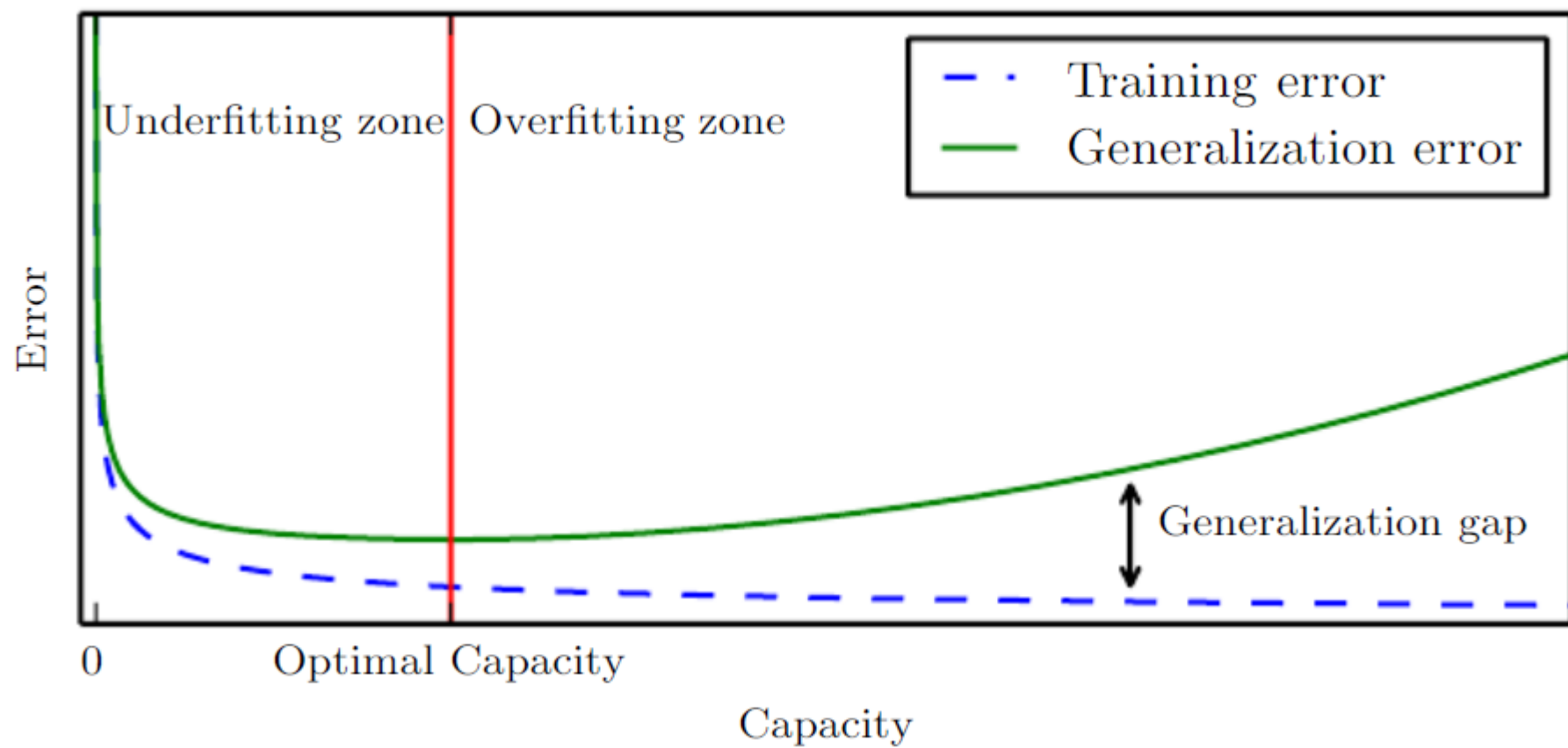
$$E_{test} = \frac{1}{M} \sum_{i=1}^M \|d_i^{test} - o(W^*, x_i^{test})\|^2$$

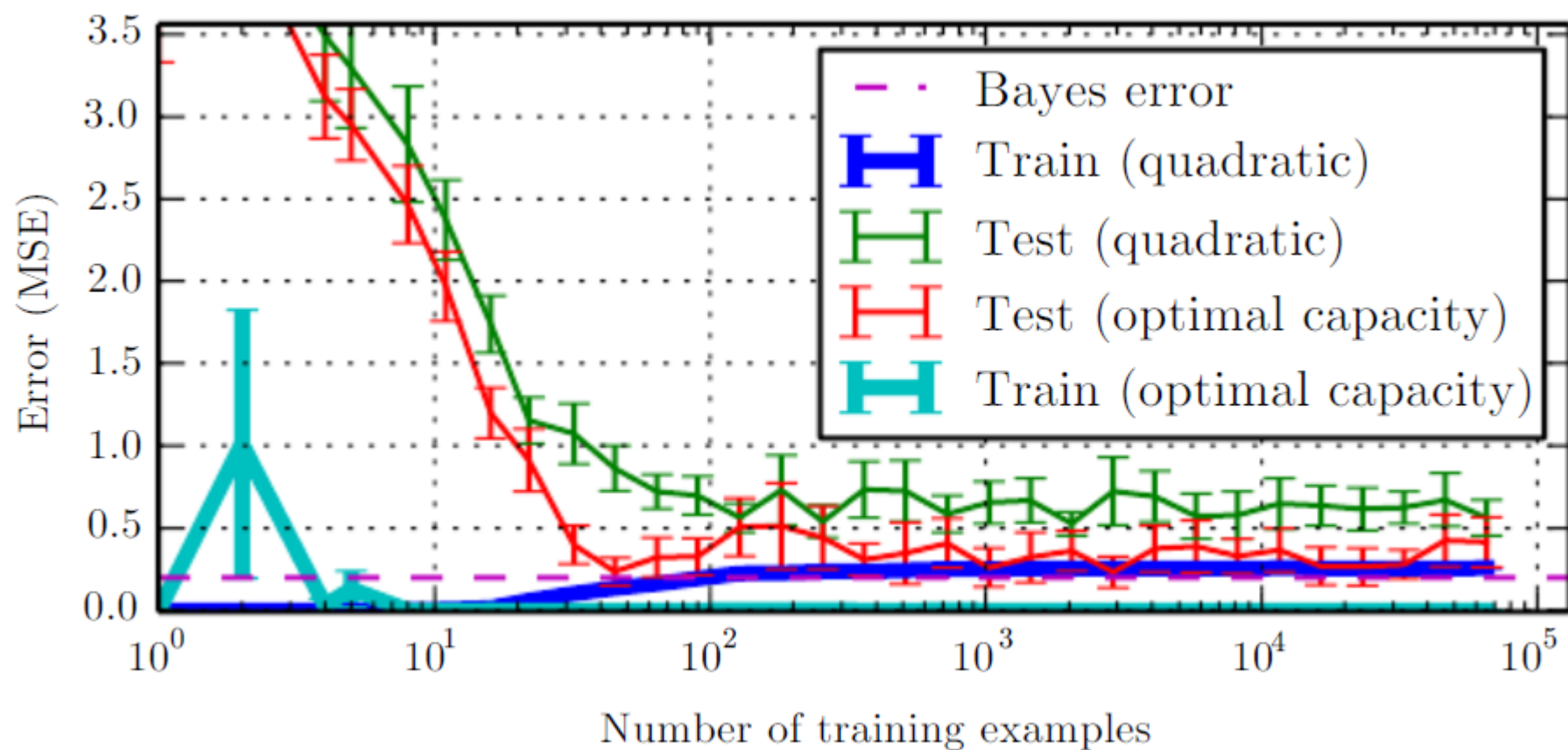
Typically, when training a machine learning model, we have access to a training set, we can compute some error measure on the training set called the **training error**, and we reduce this training error. So far, what we have described is simply an optimization problem. What separates machine learning from optimization is that we want the **generalization error**, also called the **test error**, to be low as well. The generalization error is defined as the expected value of the error on a new input. Here the expectation is taken across different possible inputs, drawn from the distribution of inputs we expect the system to encounter in practice.

1. Make the training error small.
2. Make the gap between training and test error small.

Underfitting / Overfitting

Bias-Variance dilemma





Overview

We know how to train big models to fit the training data.
Just build a big neural net and train it with backprop!
But this will just overfit the training data.

Ways to get that -

- Collect more data.

- Choose a model with the right capacity.

 - Fewer parameters: fewer hidden units and layers.

 - Weight penalties and constraints.

 - Early stopping.

- Ensemble methods - bagging, boosting, mixture of experts.

- Add Noise - Data augmentation, Stochastic neurons, Dropout.

Regularization

Models with high capacity/complexity will be able to learn weird features.
How to prevent this?

Regularization

Models with high capacity/complexity will be able to learn weird features.
How to prevent this?

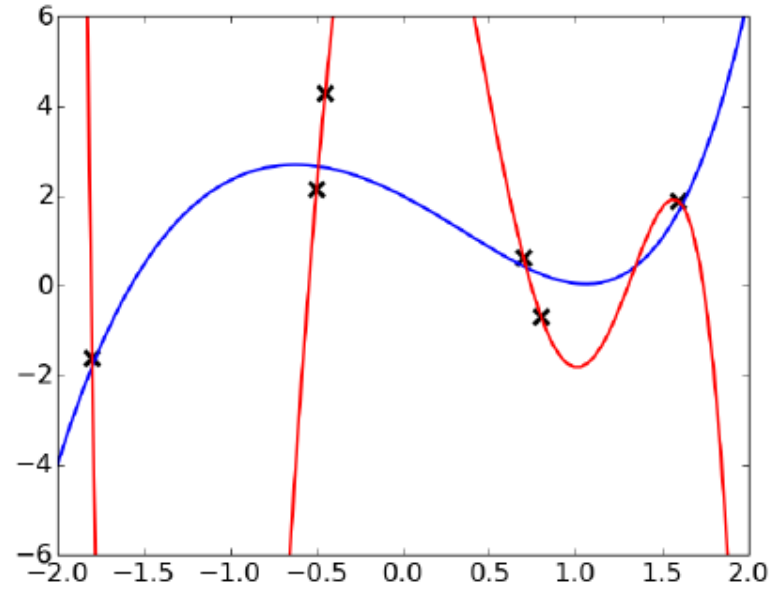
Idea: come up with a function R which is large when the features are weird and unnatural. Then try to keep R small.

New optimization problem:

$$C_{\text{reg}}(\mathbf{w}) = L(\mathbf{w}) + R(\mathbf{w}),$$

where $L(\mathbf{w})$ is the loss function (e.g. cross-entropy) and $R(\mathbf{w})$ is a regularizer.

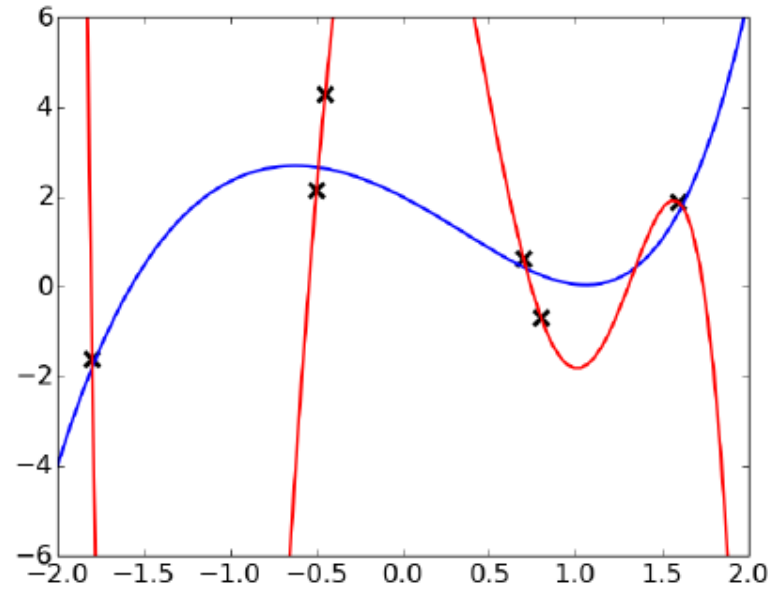
Weight penalties



$$y = 0.1x^5 + 0.2x^4 + 0.75x^3 - x^2 - 2x + 2$$

$$y = -7.2x^5 + 10.4x^4 + 24.5x^3 - 37.9x^2 - 3.6x + 12$$

Weight penalties



$$y = 0.1x^5 + 0.2x^4 + 0.75x^3 - x^2 - 2x + 2$$

$$y = -7.2x^5 + 10.4x^4 + 24.5x^3 - 37.9x^2 - 3.6x + 12$$

The red one overfits. Notice it has really large coefficients.

Weight penalties

The key idea

A network with small weights is **weaker/simpler/smooth**er than a network with big weights. If a network has small weights then it can only do simple things.

Since the model can only do simple things, it cannot learn weird and unnatural features.

Weight penalties

L_2 penalty (a.k.a L_2 decay)

$$R(\mathbf{w}) \propto \sum_i w_i^2$$

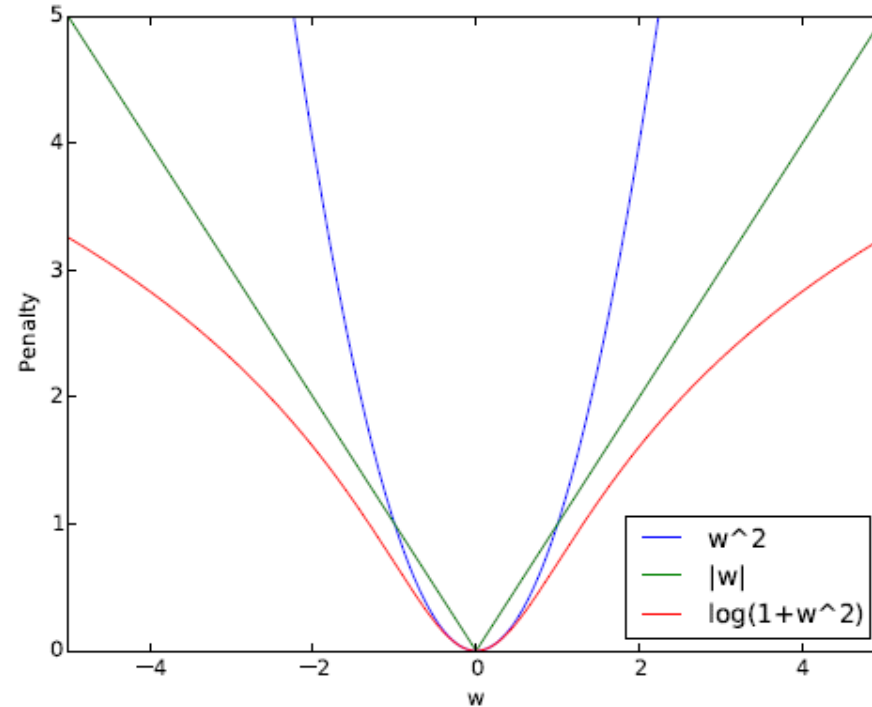
L_1 penalty (a.k.a lasso)

$$R(\mathbf{w}) \propto \sum_i |w_i|$$

Other forms of penalty:

$$R(\mathbf{w}) \propto \sum_i \log(1 + w_i^2)$$

All these regularizers would be happiest if w_i 's were all zero. They differ in how unhappy they become as w_i 's increase.



Weight constraints

Instead of just discouraging big weights, we can completely rule them out.
I.e., we can add a **hard constraint** that $\|\mathbf{w}\| \leq L$.

This keeps the weights inside a ball of radius L .

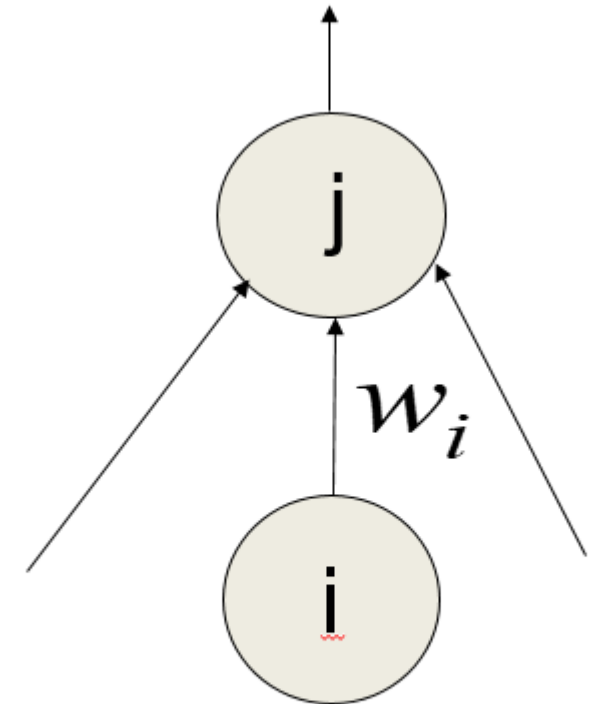
We can also view this as a regularization term:

$$R(\mathbf{w}) = \begin{cases} 0 & \|\mathbf{w}\| \leq L \\ +\infty & \text{otherwise} \end{cases}$$

L2 weight-decay via noisy inputs

- Suppose we add Gaussian noise to the inputs.
 - The variance of the noise is amplified by the squared weight before going into the next layer.
- In a simple net with a linear output unit directly connected to the inputs, the amplified noise gets added to the output.
- This makes an additive contribution to the squared error.
 - So minimizing the squared error tends to minimize the squared weights when the inputs are noisy.

$$y_j + N(0, w_i^2 \sigma_i^2)$$



$$x_i + N(0, \sigma_i^2)$$

Gaussian noise

output on one case $\longrightarrow y^{noisy} = \sum_i w_i x_i + \sum_i w_i \varepsilon_i$ where ε_i is sampled from $N(0, \sigma_i^2)$

$$\begin{aligned} E[(y^{noisy} - t)^2] &= E\left[\left(y + \sum_i w_i \varepsilon_i - t\right)^2\right] = E\left[\left((y - t) + \sum_i w_i \varepsilon_i\right)^2\right] \\ &= (y - t)^2 + E\left[2(y - t) \sum_i w_i \varepsilon_i\right] + E\left[\left(\sum_i w_i \varepsilon_i\right)^2\right] \\ &= (y - t)^2 + E\left[\sum_i w_i^2 \varepsilon_i^2\right] \quad \begin{array}{l} \text{because } \varepsilon_i \text{ is independent of } \varepsilon_j \\ \text{and } \varepsilon_i \text{ is independent of } (y - t) \end{array} \\ &= (y - t)^2 + \sum_i w_i^2 \sigma_i^2 \quad \text{So } \sigma_i^2 \text{ is equivalent to an L2 penalty} \end{aligned}$$

Question 1

Suppose we are fitting a linear regression model

$$y = wx$$

We just have one data point (x, t) , $x \neq 0$. Consider these three loss functions

$$L_1(w) = \frac{1}{2}(y - t)^2$$

$$L_2(w) = \frac{1}{2}(y - t)^2 + \frac{\lambda}{2}w^2$$

$$L_3(w) = \frac{1}{2}(y - t)^2 + \lambda|w|$$

Minimize each loss function to solve for the optimal w in terms of x, t, λ .

Note: for L_3 , you need to check 3 cases, corresponding to $w < 0, w = 0, w > 0$

What is hyperparameter tuning?

In machine learning, we need to differentiate between parameters and hyperparameters. A learning algorithm learns or estimates model parameters for the given data set, then continues updating these values as it continues to learn. After learning is complete, these parameters become part of the model. For example, each weight and bias in a neural network is a parameter.

Hyperparameters, on the other hand, are specific to the algorithm itself, so we can't calculate their values from the data. We use hyperparameters to calculate the model parameters. Different hyperparameter values produce different model parameter values for a given data set.

<https://www.anyscale.com/blog/what-is-hyperparameter-tuning>

Hyperparameter types

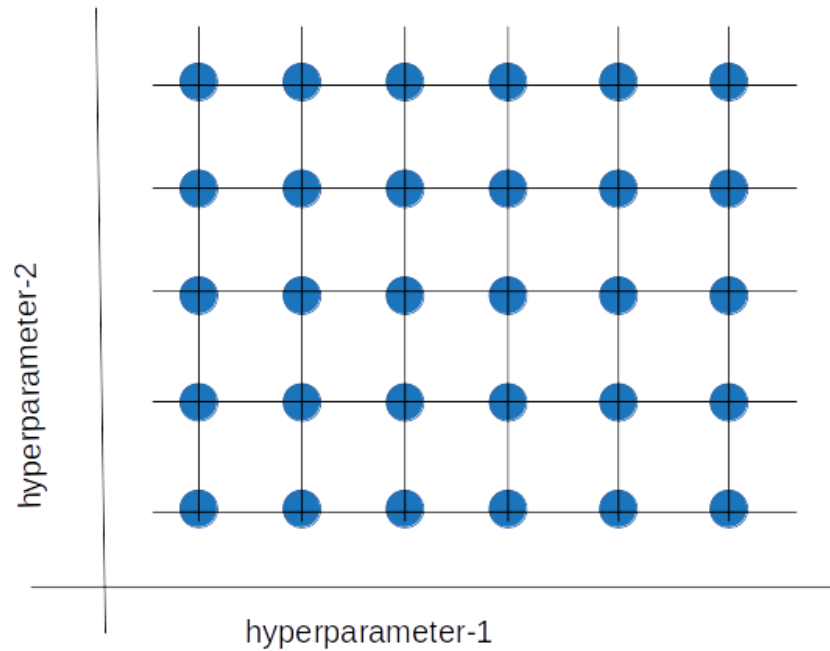
Some important hyperparameters that require tuning in neural networks are:

- **Number of hidden layers:** It's a trade-off between keeping our neural network as simple as possible (fast and generalized) and classifying our input data correctly. We can start with values of four to six and check our data's prediction accuracy when we increase or decrease this hyperparameter.
- **Number of nodes/neurons per layer:** More isn't always better when determining how many neurons to use per layer. Increasing neuron count can help, up to a point. But layers that are too wide may memorize the training dataset, causing the network to be less accurate on new data.
- **Learning rate:** Model parameters are adjusted iteratively – and the learning rate controls the size of the adjustment at each step. The lower the learning rate, the lower the changes to parameter estimates are. This means that it takes a longer time (and more data) to fit the model – but it also means that it is more likely that we actually find the minimum loss.
- **Momentum:** Momentum helps us avoid falling into local minima by resisting rapid changes to parameter values. It encourages parameters to keep changing in the direction they were *already* changing, which helps prevent zig-zagging on every iteration. Aim to start with low momentum values and adjust upward as needed.

Methods for tuning hyperparameters

Grid search

Grid search is a sort of “brute force” hyperparameter tuning method. We create a grid of possible discrete hyperparameter values then fit the model with every possible combination. We record the model performance for each set then select the combination that has produced the best performance.

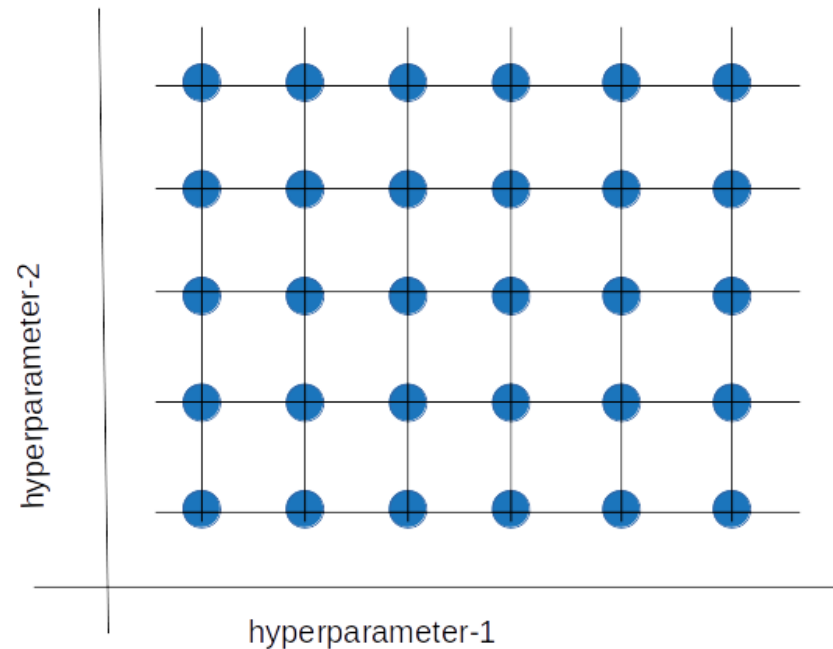


Grid search is a hyperparameter tuning method in which we create a grid of possible discrete hyperparameter values, then fit the model with every possible combination.

Methods for tuning hyperparameters

Grid search

Grid search is a sort of “brute force” hyperparameter tuning method. We create a grid of possible discrete hyperparameter values then fit the model with every possible combination. We record the model performance for each set then select the combination that has produced the best performance.



Grid search is a hyperparameter tuning method in which we create a grid of possible discrete hyperparameter values, then fit the model with every possible combination.

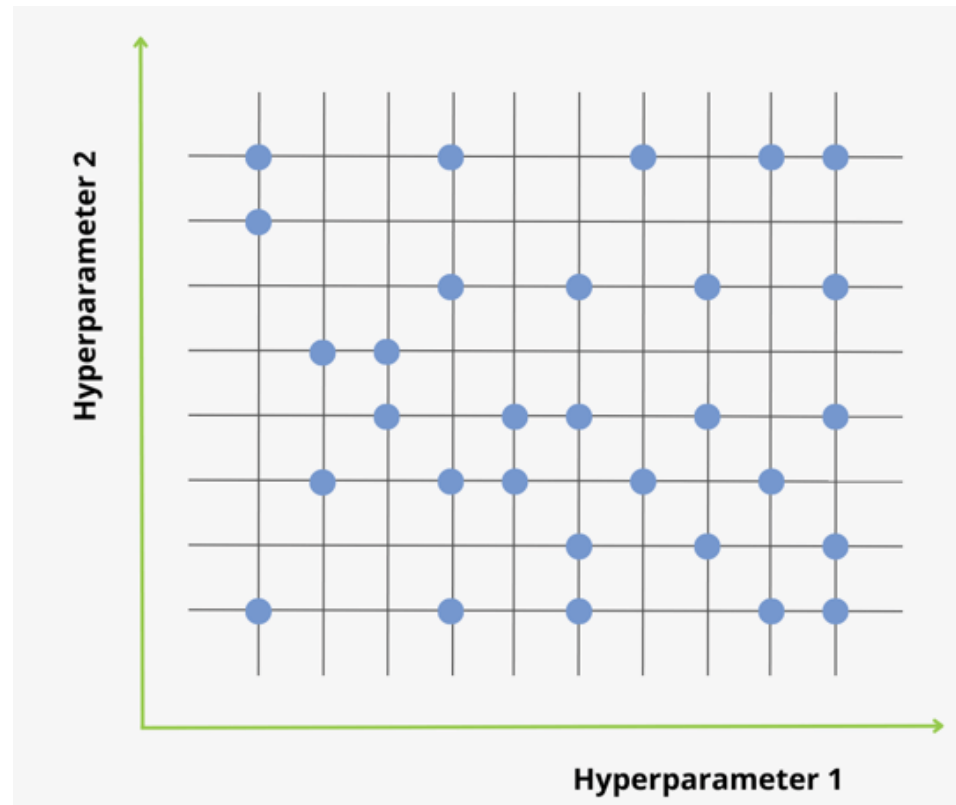
C	0.5	0.701	0.703	0.697	0.696
	0.4	0.699	0.702	0.698	0.702
	0.3	0.721	0.726	0.713	0.703
	0.2	0.706	0.705	0.704	0.701
	0.1	0.698	0.692	0.688	0.675
		0.1	0.2	0.3	0.4
		Alpha			

Randomised search

In case of randomised search, unlike grid search, not all given parameter values are tried out.

Hence, a fixed number of parameter settings is sampled from the specified distribution of values.

For each hyperparameter, either a distribution of possible values or a list of discrete values (to be sampled uniformly) can be specified.



Informed Method: Coarse to Fine Tuning

A basic informed search methodology.

The process follows:

1. Random search
2. Find promising areas in the search space
3. Grid search in the smaller area
4. Continue until optimal score is obtained

<https://www.kdnuggets.com/2020/11/algorithms-for-advanced-hyper-parameter-optimization-tuning.html>

You could substitute (3) with random searches before the grid search.

Why Coarse to Fine?

Coarse to Fine tuning optimizes and uses the advantages of both grid and random search.

- Wide searching capabilities of random search
- Deeper search once you know where a good spot is likely to be

No need to waste time on search spaces that are not giving good results !! Therefore, this better utilizes the spending of time and computational efforts, i.e we can iterate quickly, also there is boost in the performance.

▼ Bayesian hyperparameter tuning:

When it comes to using Bayesian principles in hyperparameter tuning the following steps are generally followed:

- Pick a combination of hyperparameter values (our belief) and train the machine learning model with it.
- Get the evidence (i.e. score of the model)
- Update our belief that can lead to model improvement.

Terminate when a stopping criteria is met (think when a loss a quantity is minimized or classification accuracy is maximized).

To understand this in a bit more details, let's introduce the Bayes rule which is 250+ years old. It helps us to predict Y given X, what is the probability of John playing soccer tomorrow provided it rains today, for example. So, the Y here is the probability of John playing soccer tomorrow and X denotes it rained today. In probabilistic literature, you can denote it like so -

$$P(Y|X)$$

https://wandb.ai/wandb_fc/articles/reports/Bayesian-Hyperparameter-Optimization-A-Primer--VmIldzo1NDQyNzcw

$$P(Y|X)$$

It read as - what is the probability of Y given X. Now, this has got an RHS as well:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

where,

- $P(X)$ is the probability of observing this new evidence.
- $P(X|Y)$ is the probability of observing the new evidence X given the event Y that we care about.
- $P(Y)$ is the initial hypothesis about the event Y that we care about (treat it like an initial belief about the event Y).

So, if we try to apply the same principle to hyperparameter tuning, following would be the equation for it -

$$P(\text{metric}|\text{hyperparameter combination}) = \frac{P(\text{hyperparameter combination}|\text{metric})P(\text{metric})}{P(\text{hyperparameter combination})}$$

So, now each of the above terms would be:

- $P(\text{metric} | \text{hyperparameter combination})$ gives the probability of the given metric to be minimized/maximized given the combination of hyperparameter values.
- $P(\text{hyperparameter combination} | \text{metric})$ is the probability of a certain hyperparameter combination if the given metric is minimized/maximized.
- $P(\text{metric})$ is the initial metric quantity in scalar.
- $P(\text{hyperparameter combination})$ is the probability of getting that particular hyperparameter combination.

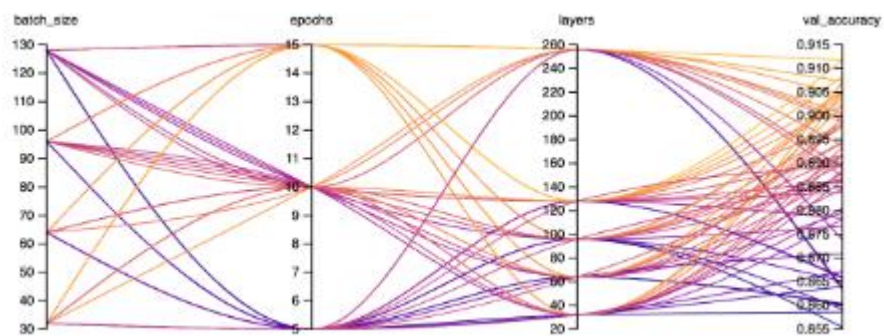
Selection Function

The selection function is the criteria by which the next set of hyperparameters are chosen from the surrogate function. The most common choice of criteria is Expected Improvement:

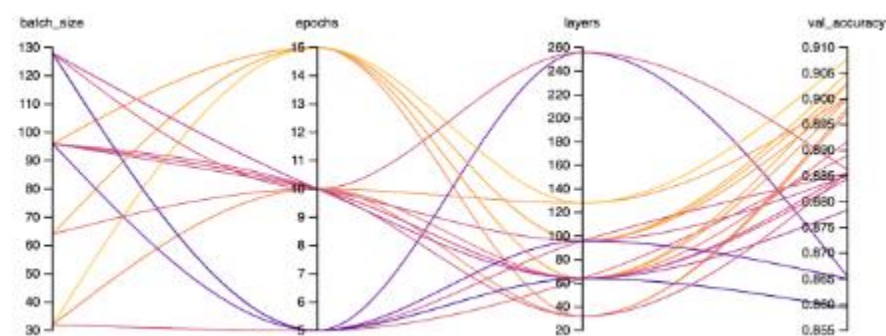
$$EI_{y^*}(x) = \int_{-\infty}^{y^*} (y^* - y)p(y|x)dy$$

Here y^* is a threshold value of the objective function, x is the proposed set of hyperparameters, y is the actual value of the objective function using hyperparameters x , and $p(y | x)$ is the surrogate probability model expressing the probability of y given x . If that's all a little much, in simpler terms, **the aim is to maximize the Expected Improvement with respect to x** . This means finding the best hyperparameters under the surrogate function $p(y | x)$.

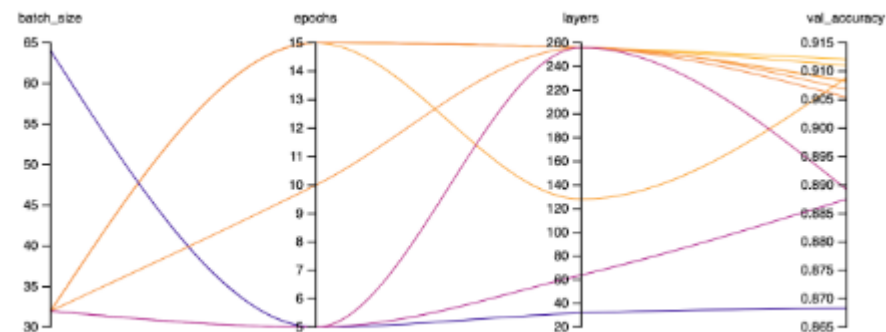
<https://towardsdatascience.com/a-conceptual-explanation-of-bayesian-model-based-hyperparameter-optimization-for-machine-learning-b8172278050f>



Grid Search



Random Search



Bayesian Search

Cross Validation

A problem with having a validation set-

- We never get to train on it.

- If we have only a small amount of precious labelled data, it is not ok to just leave out part of it.

Cross Validation

A problem with having a validation set-

- We never get to train on it.

- If we have only a small amount of precious labelled data, it is not ok to just leave out part of it.

Cross Validation

- Split the dataset into N chunks.

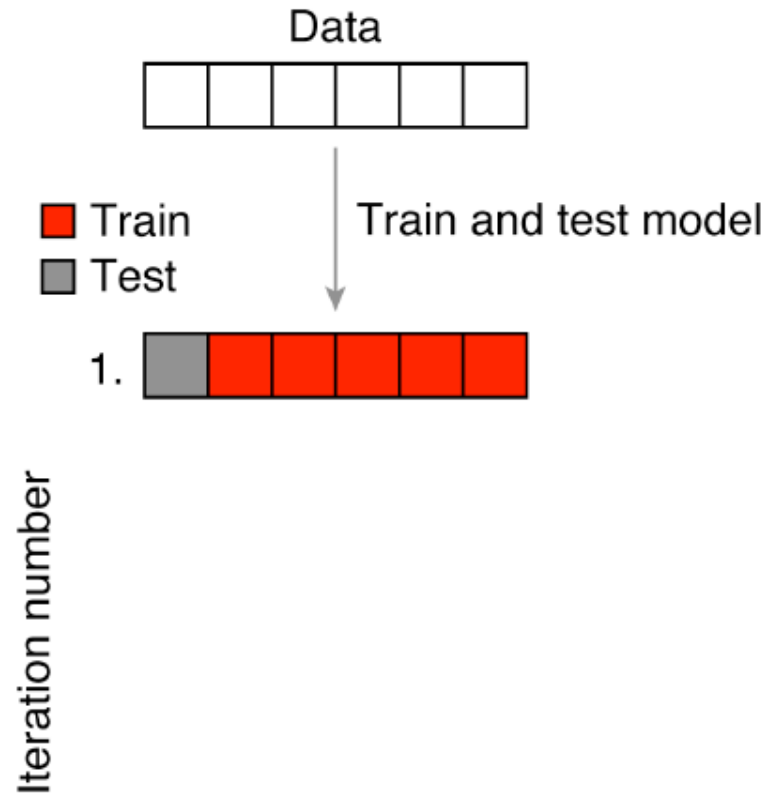
- Train N different models.

- For each model pick a different chunk for validation, use the other $N - 1$ chunks for training.

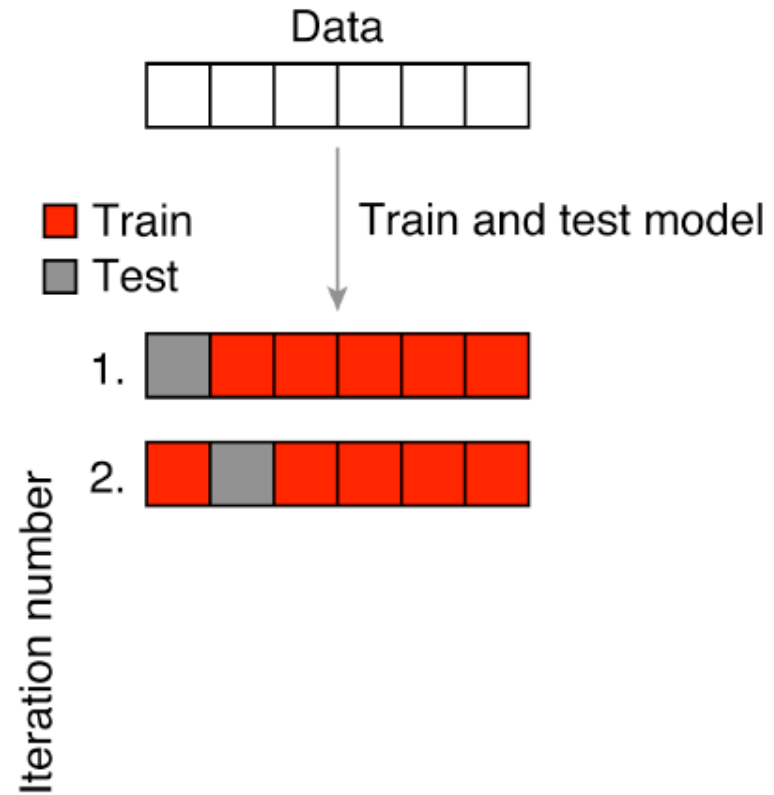
- So, we get to train on everything, but not in the same model.

- Use the average validation performance to make decisions about hyperparameters.

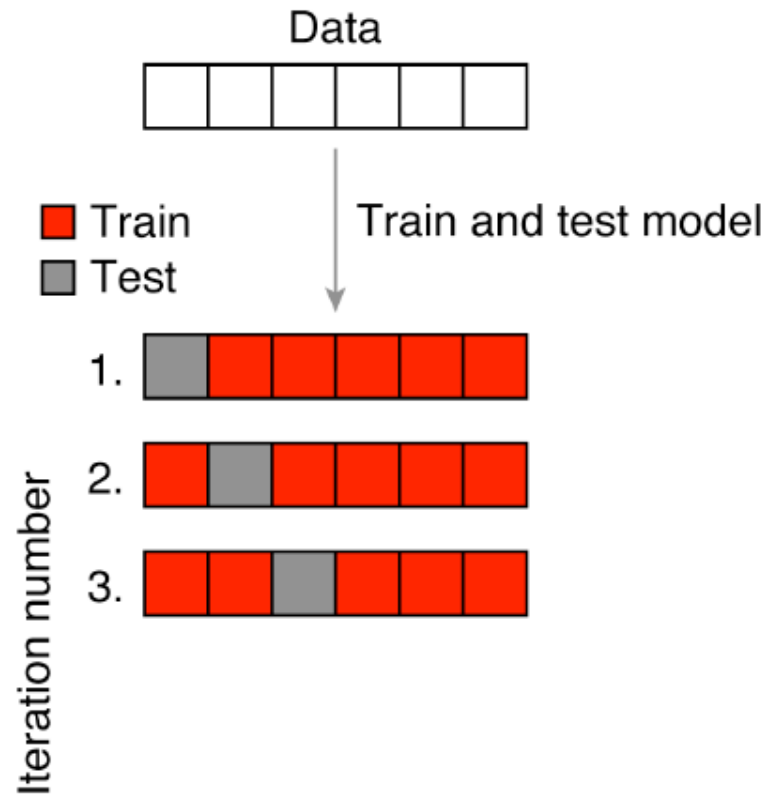
Leave-one-out cross-validation



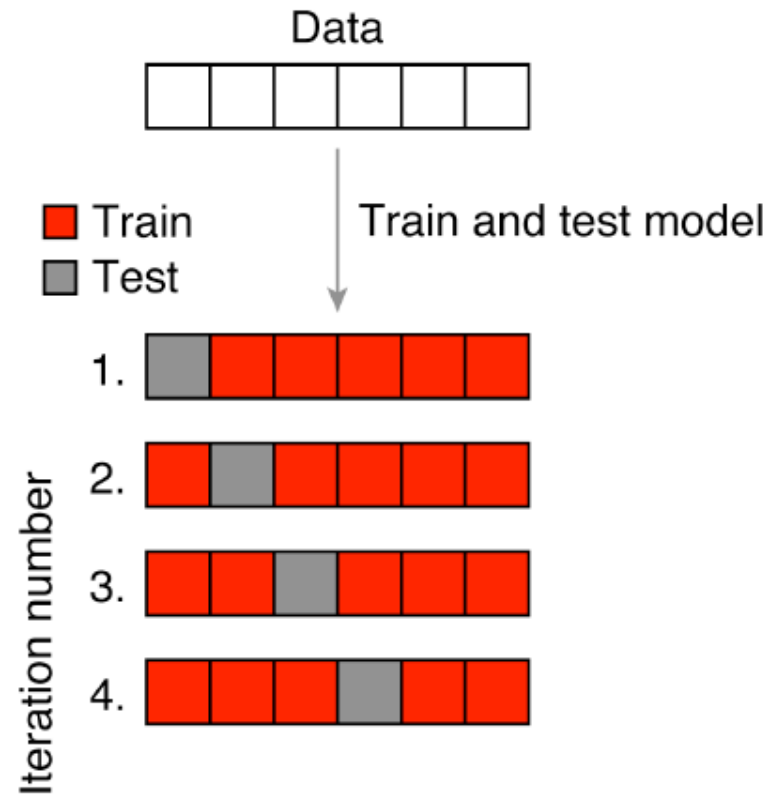
Leave-one-out cross-validation



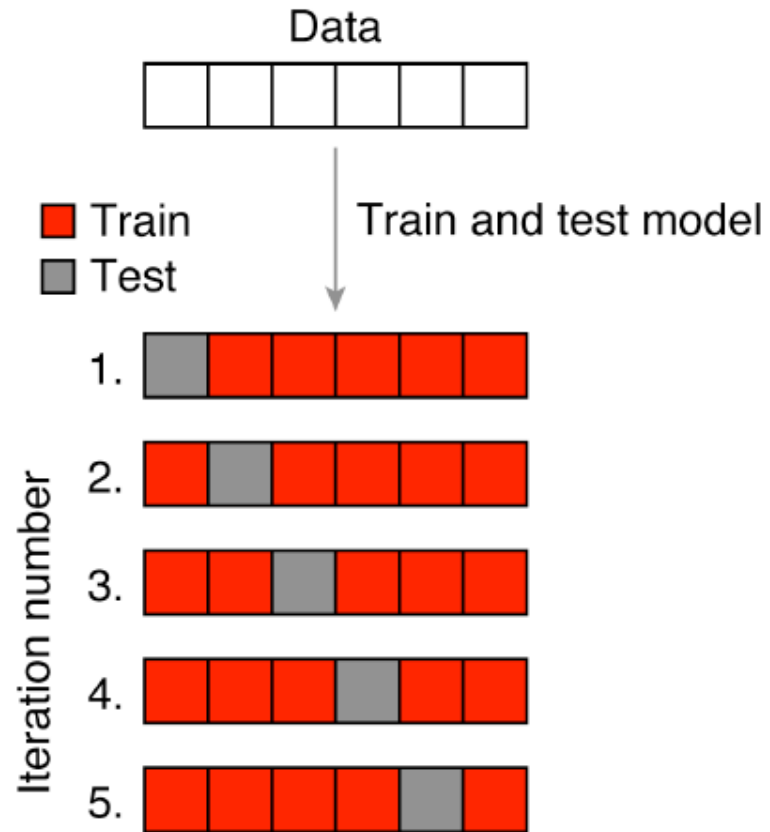
Leave-one-out cross-validation



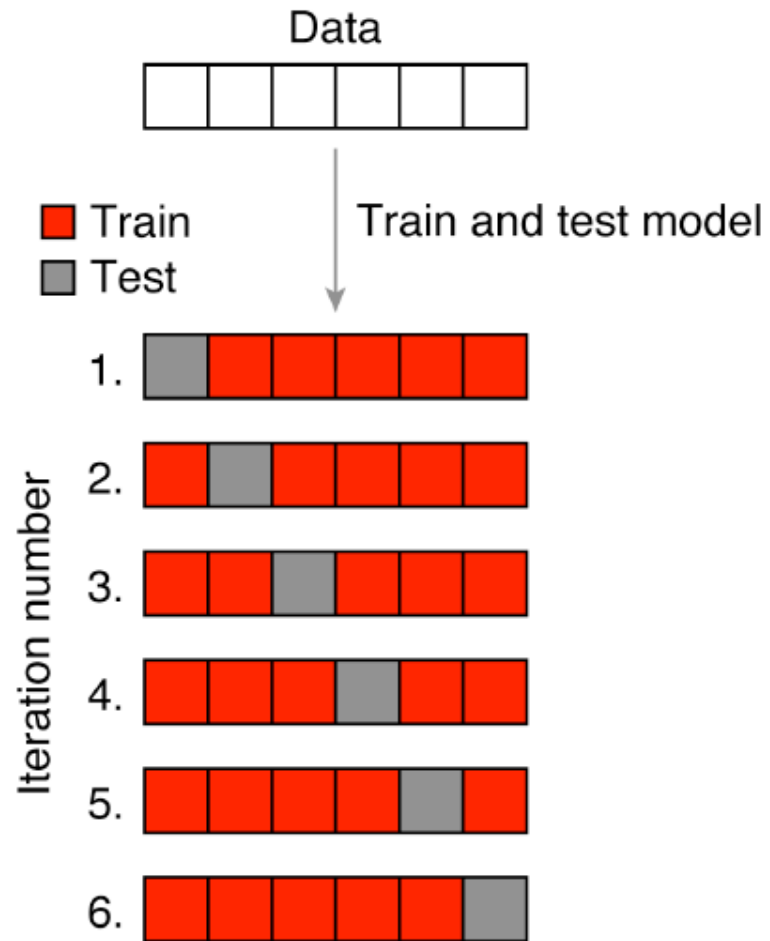
Leave-one-out cross-validation



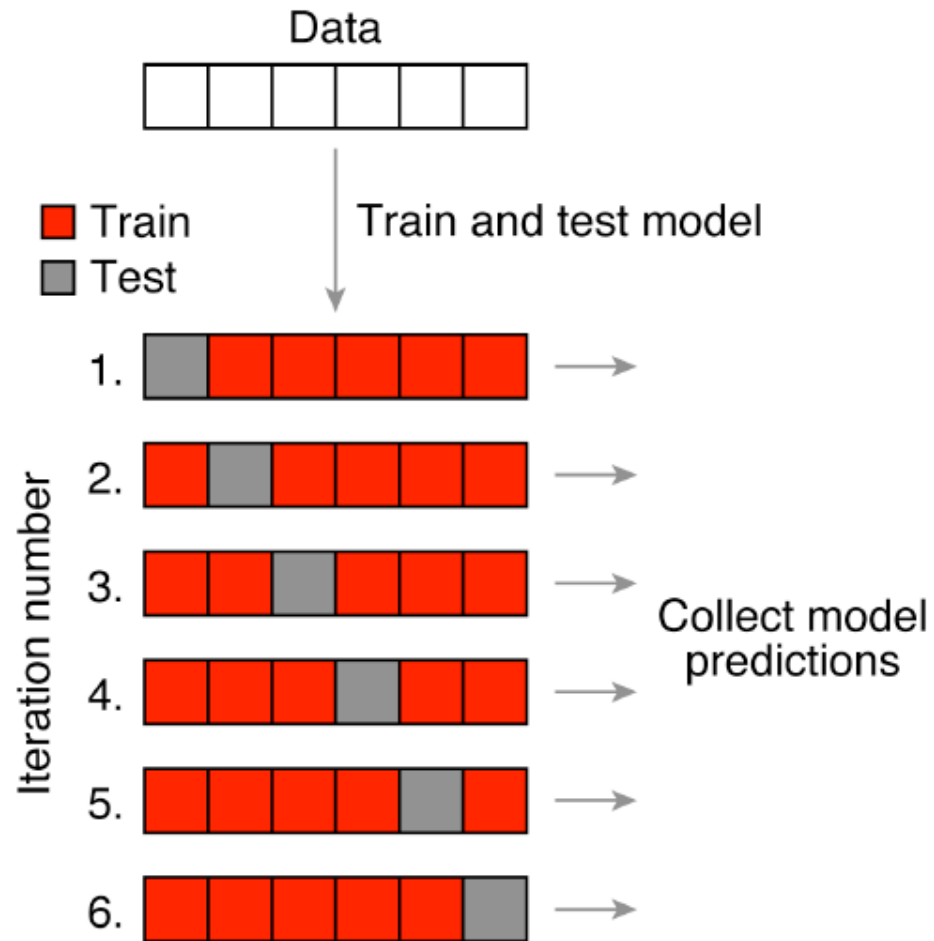
Leave-one-out cross-validation



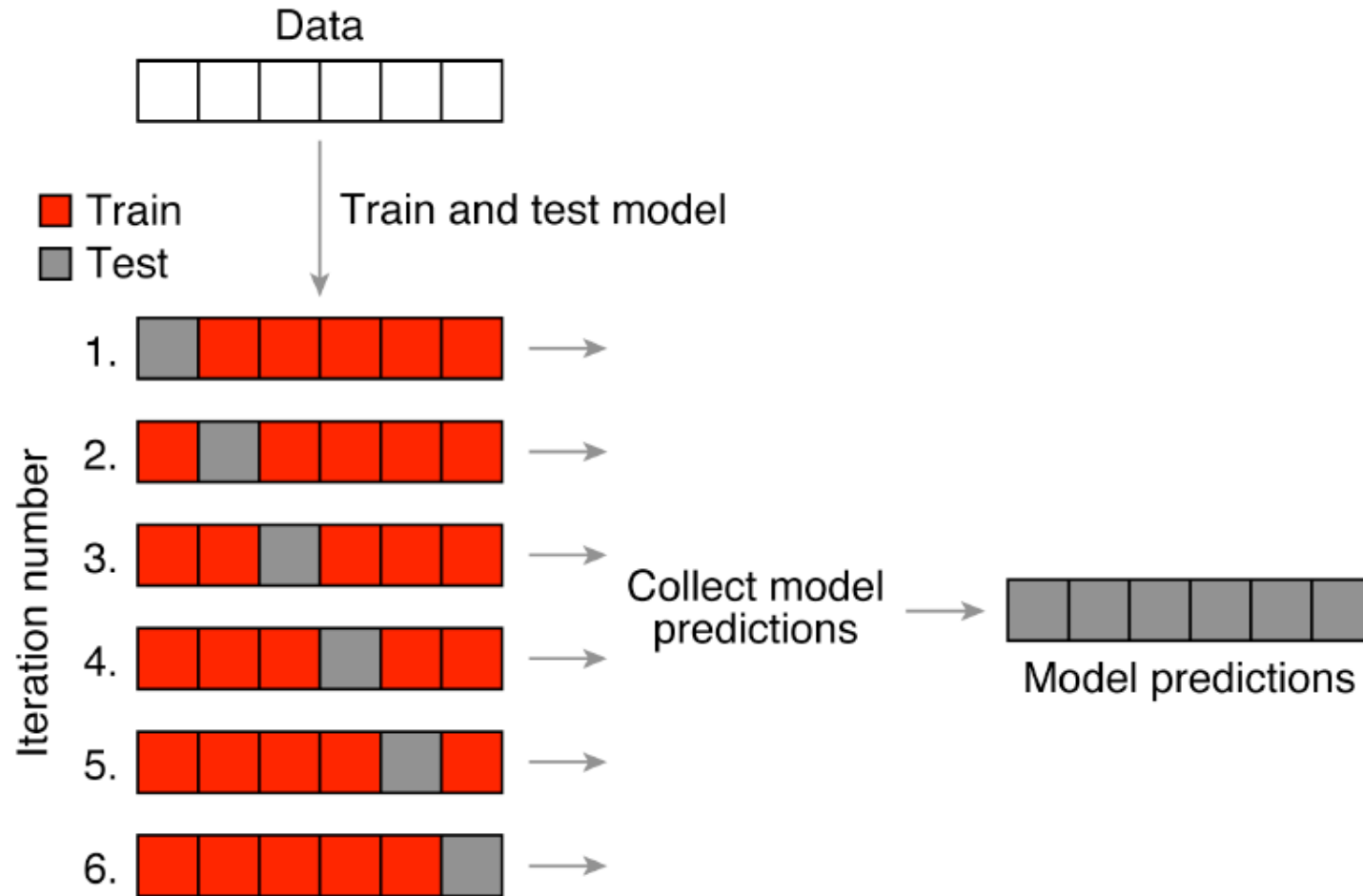
Leave-one-out cross-validation



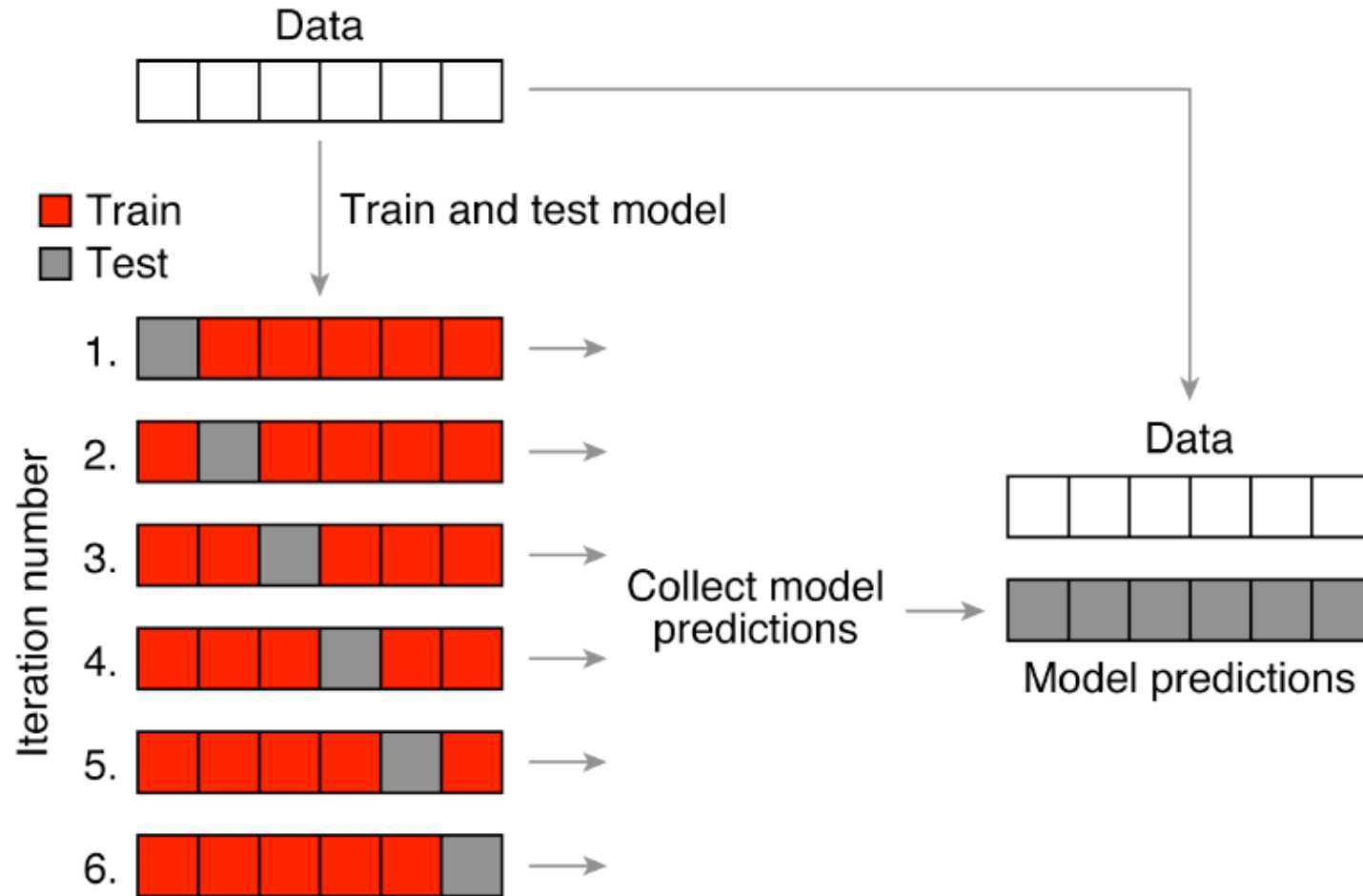
Leave-one-out cross-validation



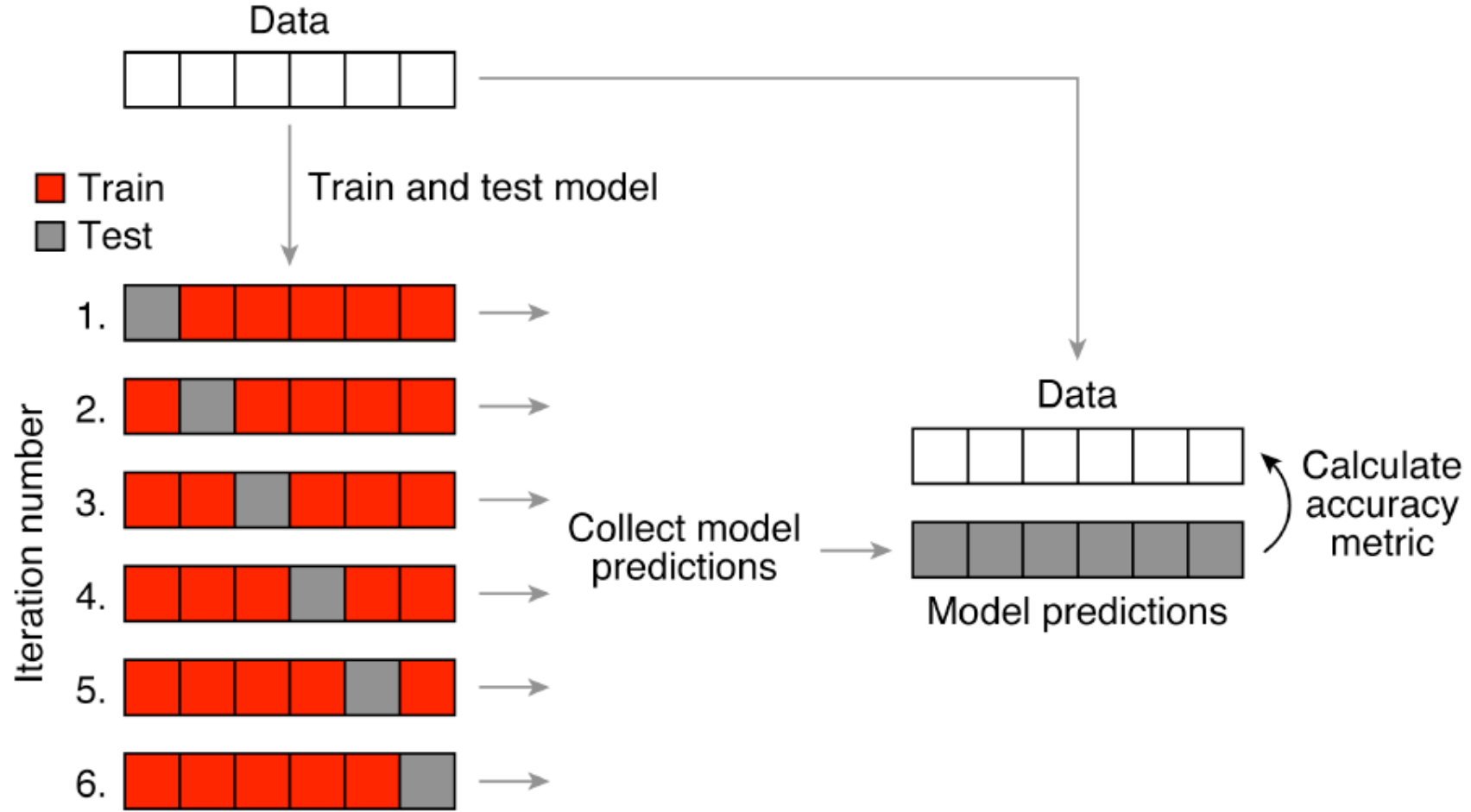
Leave-one-out cross-validation



Leave-one-out cross-validation



Leave-one-out cross-validation



K-fold Cross-Validation

Steps:

1. Split training data into K equal parts
2. Fit the model on k-1 parts and calculate test error using the fitted model on the kth part
3. Repeat k times, using each data subset as the test set once. (usually k= 5~20)



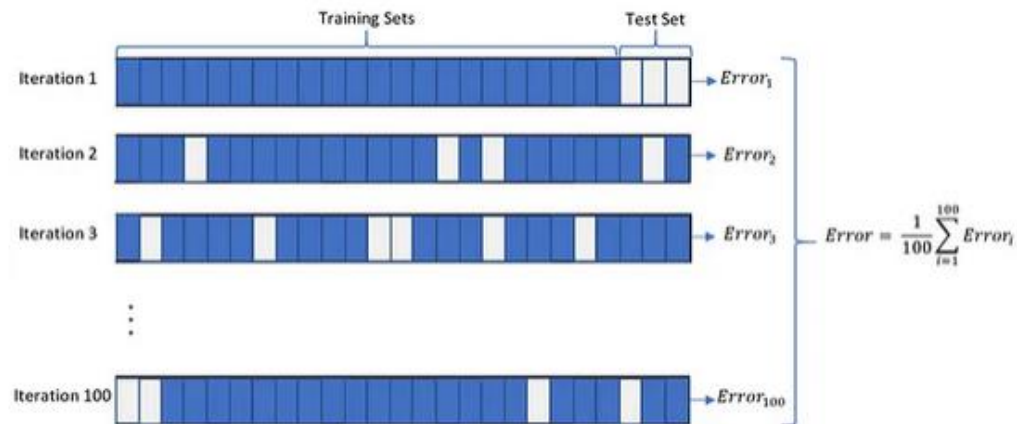
Monte Carlo Cross-Validation

Also known as repeated random subsampling CV

Steps:

1. Split training data randomly (maybe 70–30% split or 62.5–37.5% split or 86.3–13.7%split). For each iteration, the train-test split percentage is different.
2. Fit the model on train data set for that iteration and calculate test error using the fitted model on test data
3. Repeat many iterations (say 100 or 500 or even 1000 iterations) and take the average of the test errors.

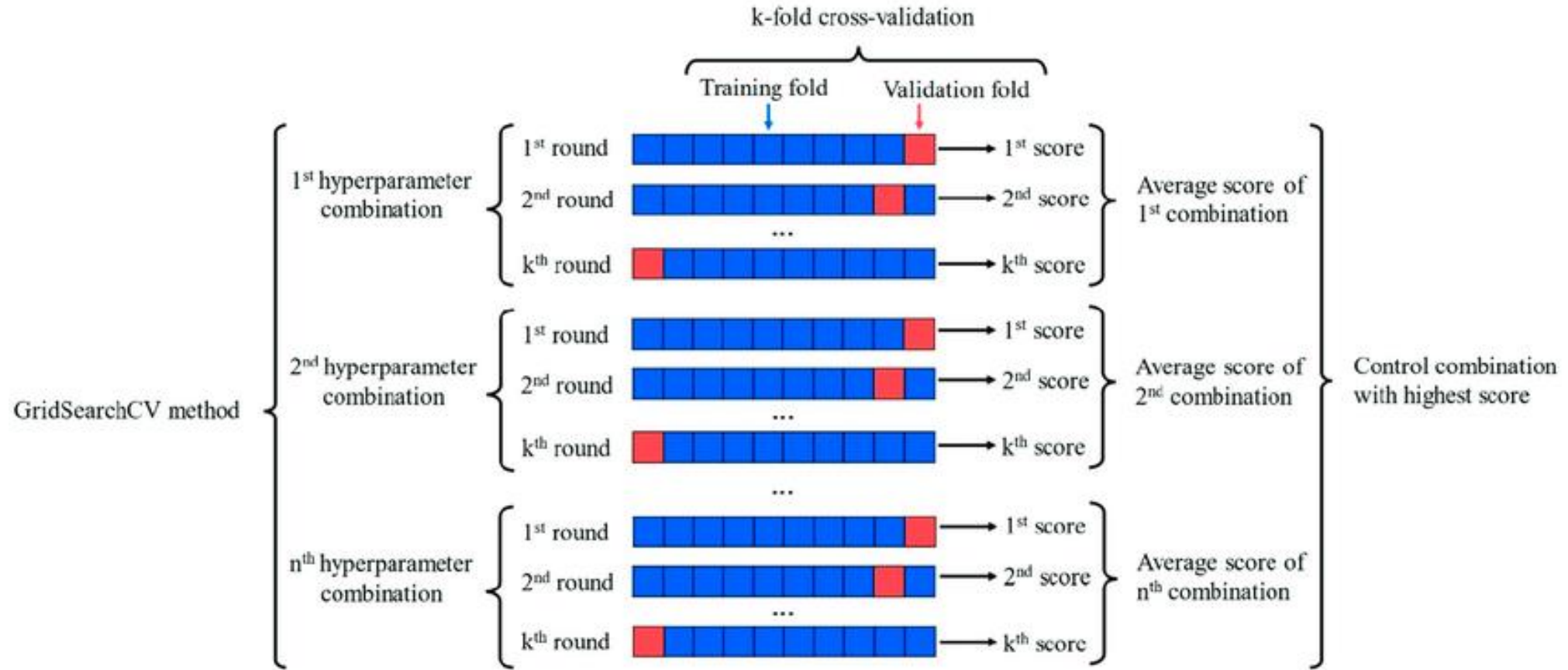
Note - the same data can be selected more than once in the test set or even never at all.



Monte Carlo CV, Iterations = 100

Differences between the two methods

K Fold CV	Monte Carlo CV
Each data is tested exactly <i>Once</i>	Each data is tested arbitrary times (even 0)
Number of partitions limited by k	Large number of possible partitions
Results unbiased but high variance	Higher bias but low variance



Data Augmentation

Having more data is always better.

If we don't have more data, lets create some using what we have!

Data Augmentation

Having more data is always better.

If we don't have more data, let's create some using what we have!

For images :

- Translate: Train on random crops of the image.

- Flip horizontally.

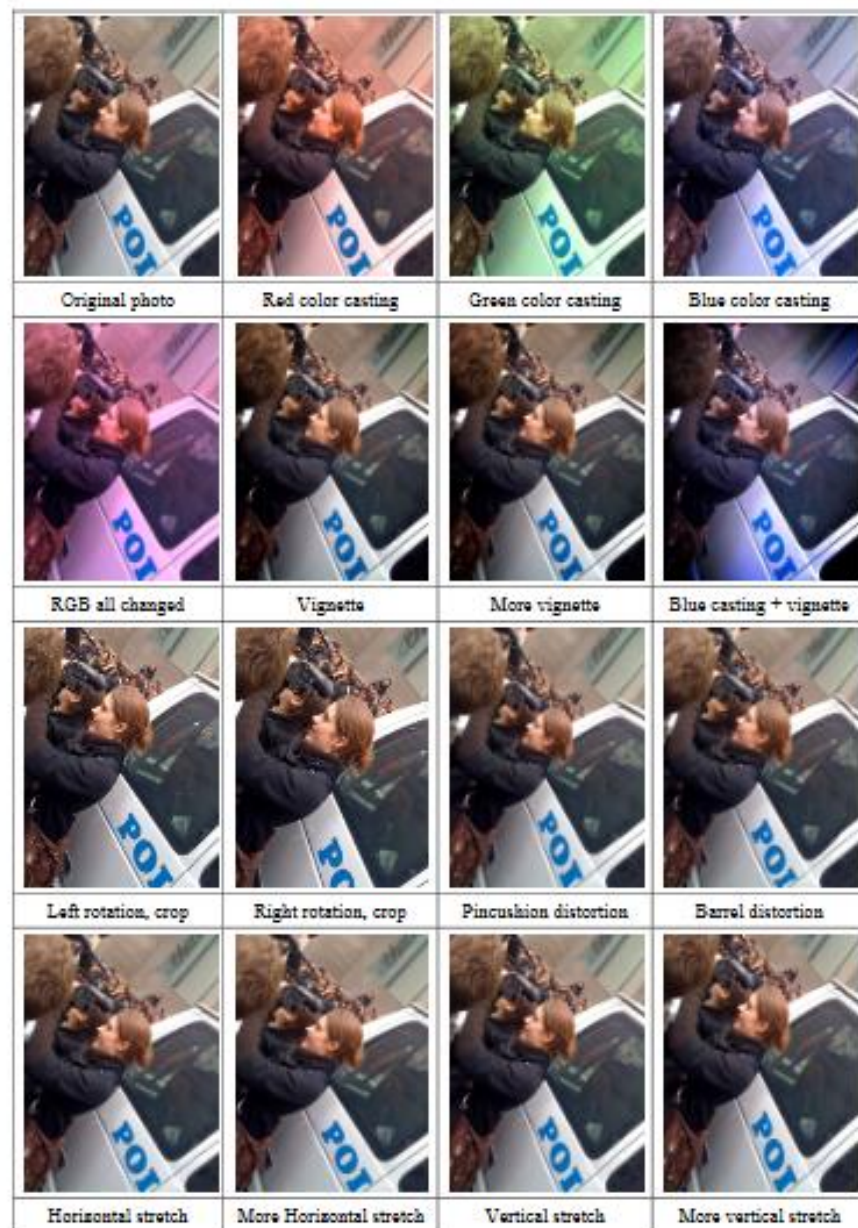
- Add noise along eigen vectors of the color space.

- Add Distortions : Stretch or squash the image.

For speech : VTLP (Vocal Tract Length Perturbation).

Just add plain Gaussian Noise or salt-and-pepper noise.

This data will not be as good as new data, but it's better than nothing.



<https://arxiv.org/pdf/1501.02876v2.pdf?>

Figure 5. Effects of data augmentation.

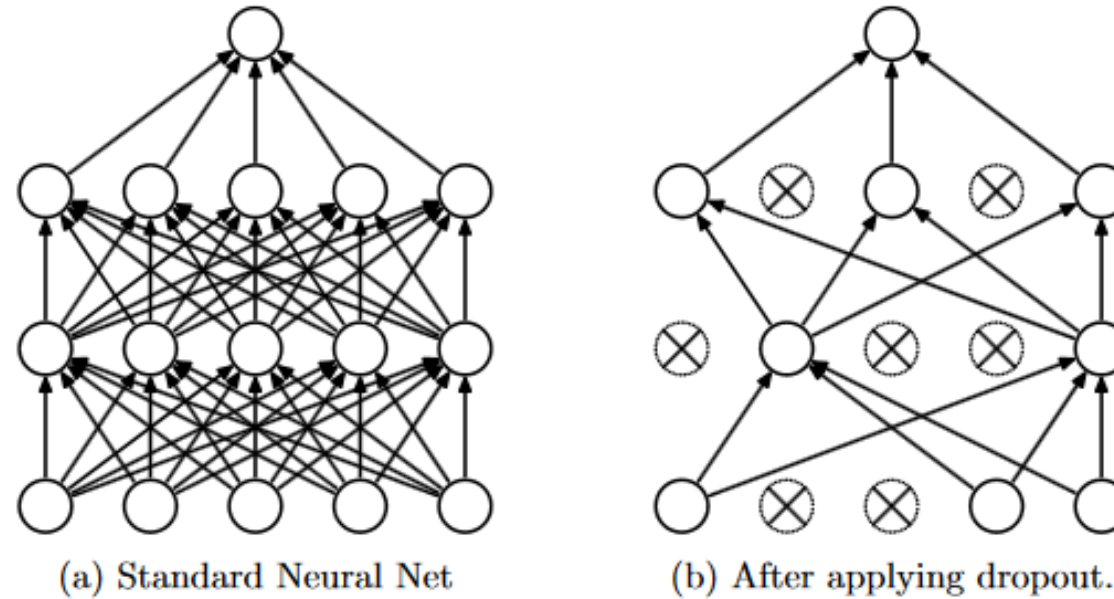


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

<https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>

DROPOUT

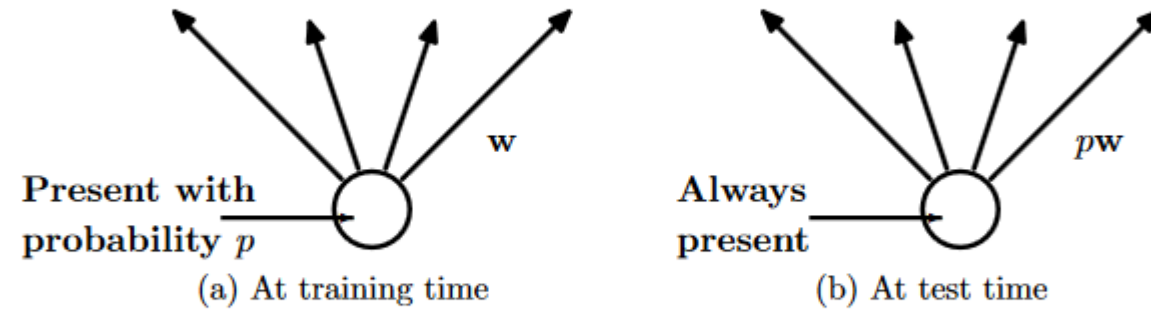
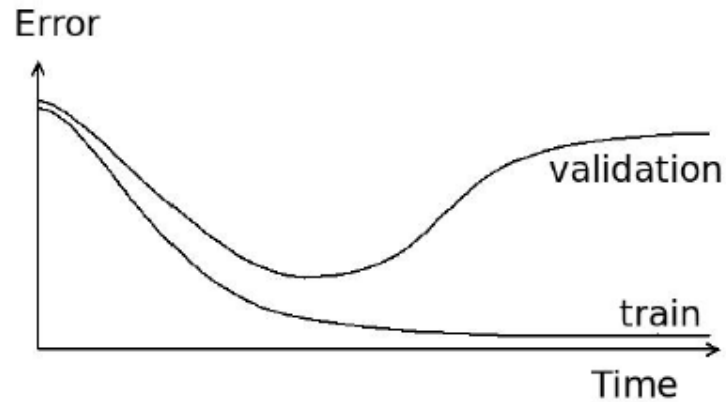


Figure 2: **Left:** A unit at training time that is present with probability p and is connected to units in the next layer with weights w . **Right:** At test time, the unit is always present and the weights are multiplied by p . The output at test time is same as the expected output at training time.

<https://jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf>

Early stopping

Hold out a part of the training set. This is called a validation set.
Stop training when performance on the validation set starts getting worse.



Picture credits: Ilya Sutskever <https://theneural.wordpress.com/>

Use this to

- Prevent overfitting.

- Make decisions about network architectures and tuning parameters :

- Pick the model that gets the lowest validation error.



PERGAMON

Neural Networks 11 (1998) 761–767

Neural
Networks

Contributed article

Automatic early stopping using cross validation: quantifying the criteria

Lutz Prechelt*

Fakultät für Informatik, Universität Karlsruhe, Karlsruhe, Germany

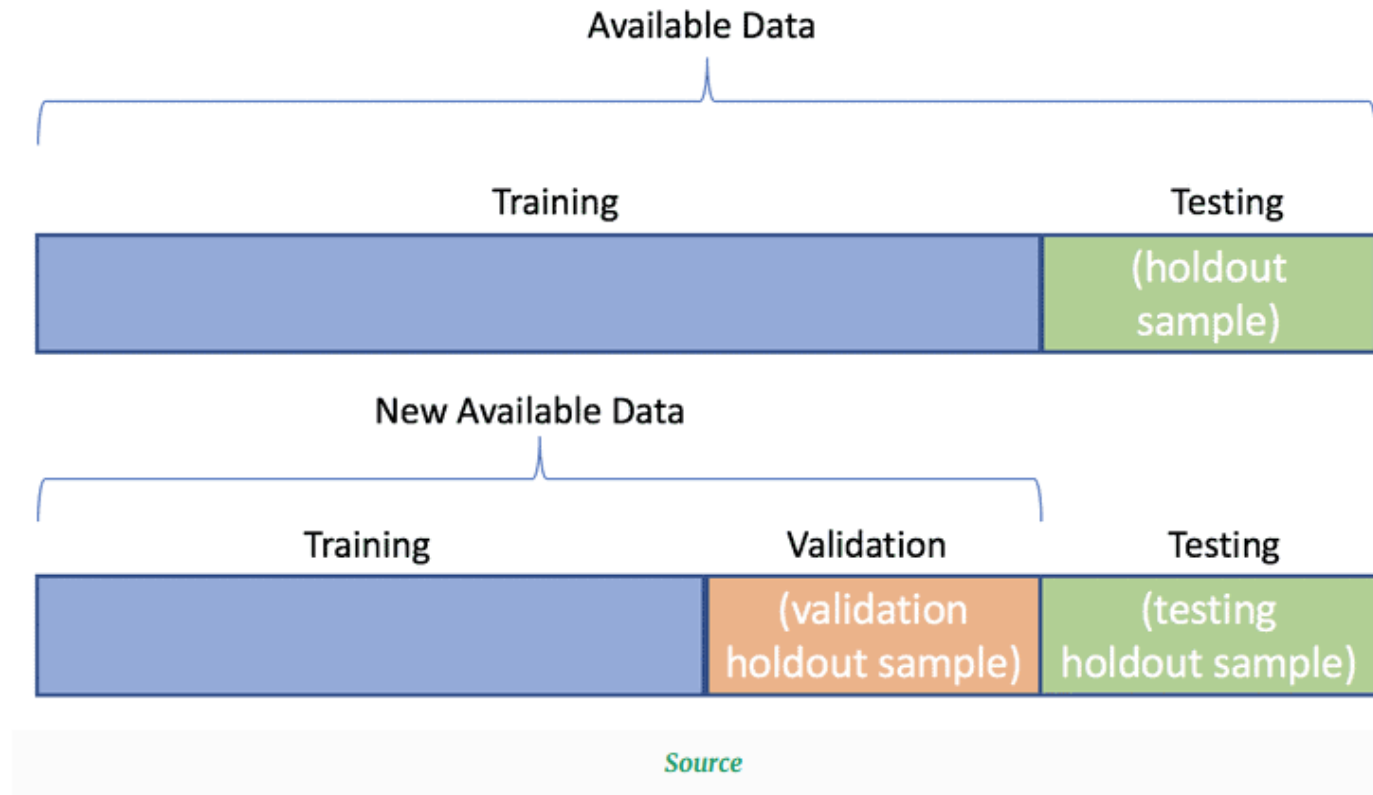
Received 28 May 1995; accepted 3 November 1997

Abstract

Cross validation can be used to detect when overfitting starts during supervised training of a neural network; training is then stopped before convergence to avoid the overfitting ('early stopping'). The exact criterion used for cross validation based early stopping, however, is chosen in an ad-hoc fashion by most researchers or training is stopped interactively. To aid a more well-founded selection of the stopping criterion, 14 different automatic stopping criteria from three classes were evaluated empirically for their efficiency and effectiveness in 12 different classification and approximation tasks using multi-layer perceptrons with RPROP training. The experiments show that, on average, slower stopping criteria allow for small improvements in generalization (in the order of 4%), but cost about a factor of 4 longer in training time. © 1998 Elsevier Science Ltd. All rights reserved.

Keywords: Early stopping; Overfitting; Cross validation; Generalization; Empirical study; Supervised learning

<https://www.sciencedirect.com/science/article/pii/S0893608098000100>



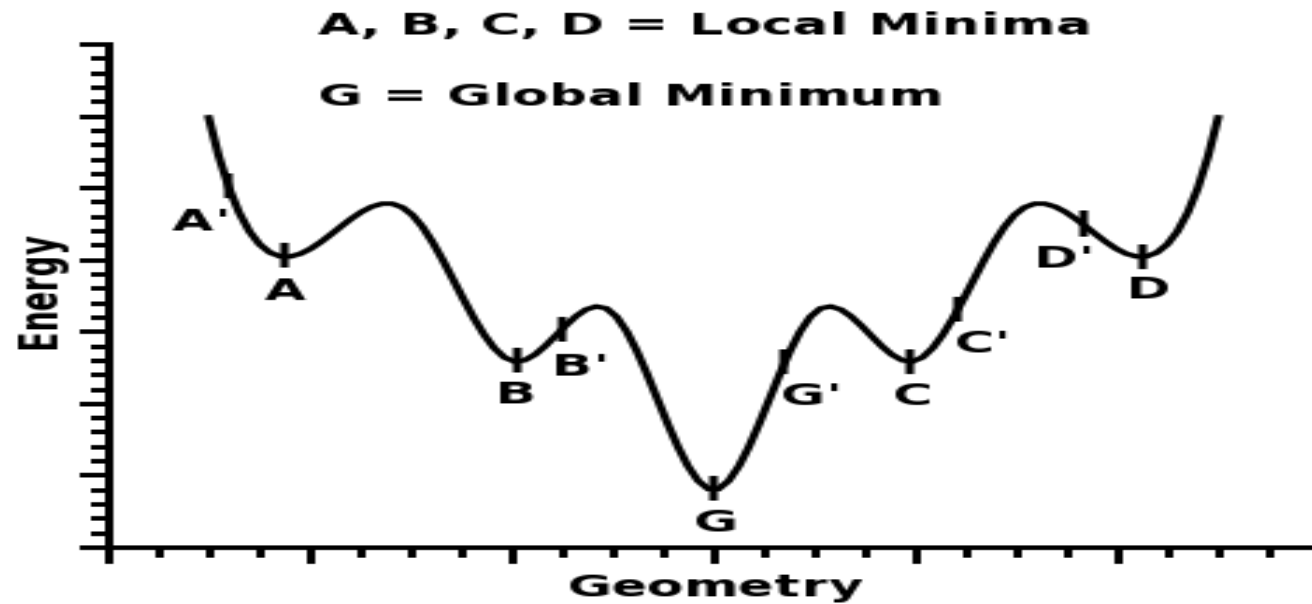
Data splitting is the process of splitting data into 3 sets:

- Data which we use to design our models (Training set)
- Data which we use to refine our models (Validation set)
- Data which we use to test our models (Testing set)

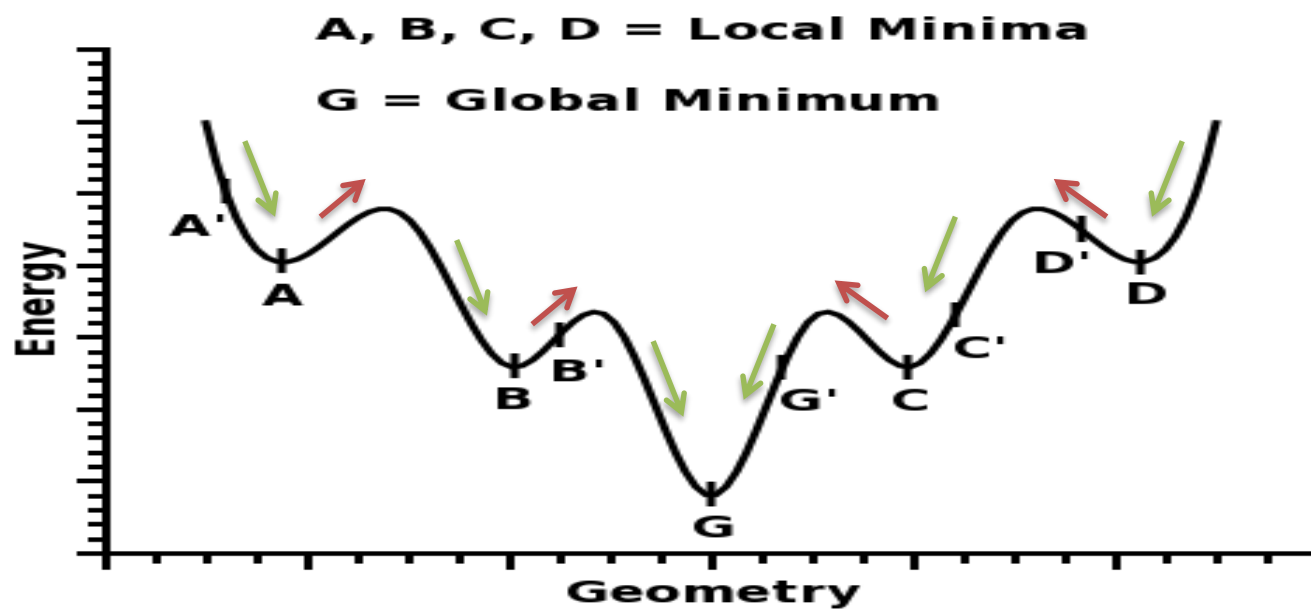
<https://algotrading101.com/learn/train-test-split/>

- **Simulated Annealing**

- Annealing : treating melted metals with heat, to control the final form after cooling...
- Glass forming, forging swords, etc....
- Related to escaping from **LOCAL MINIMUM**, and finding **GLOBAL MINIMUM**



- Problem with gradient descent :

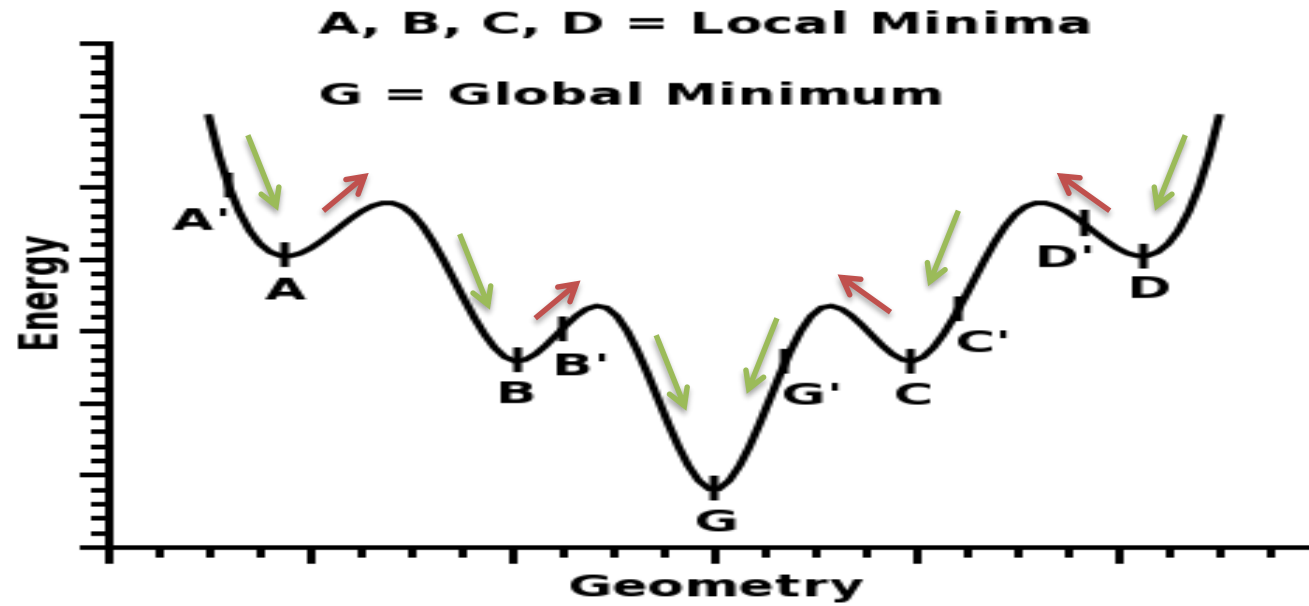


GOING DOWNHILL : Allowed



GOING UPHILL : Not allowed

- Problem with gradient descent :



GOING DOWNHILL : Allowed



GOING UPHILL : Not allowed

- IDEA : What if we allow GOING UPHILL with a certain probability?

PROBLEM :

$$\min_x E(x)$$

PROBLEM :

$$\min_x E(x)$$



- Choose initial TEMPERATURE (high)
- Choose $x(0)$, Evaluate $E(x(0))$
- $\vdots$
- Search a new point $x(i)$. Evaluate $E(x(i))$
- Accept $x(i)$ with a probability

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$

- Lower the temperature (COOLING)
- Repeat

PROBLEM :

$$\min_x E(x)$$

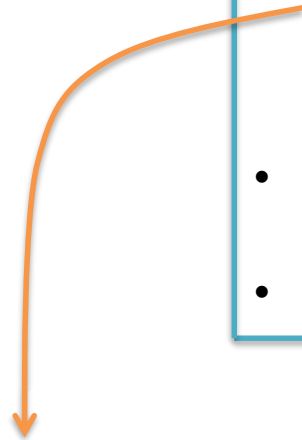
http://en.wikipedia.org/wiki/Simulated_annealing



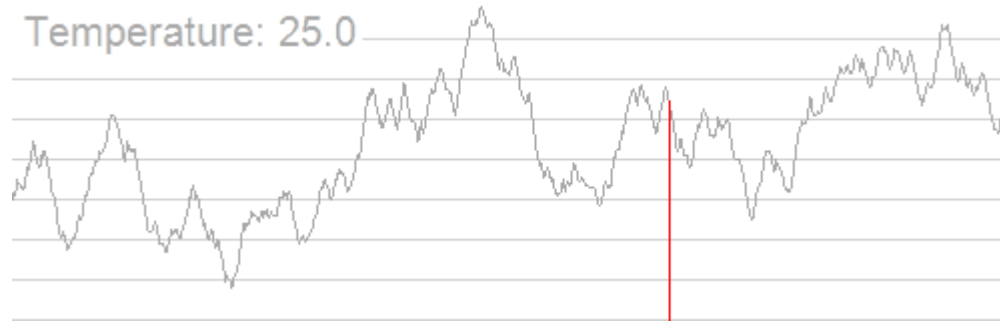
- Choose initial TEMPERATURE (high)
- Choose $x(0)$, Evaluate $E(x(0))$
- $\vdots$
- Search a new point $x(i)$. Evaluate $E(x(i))$
- Accept $x(i)$ with a probability

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$

- Lower the temperature (COOLING)
- Repeat



- Let $r = \text{rand}(0,1)$, uniformly distributed
- If $r < p$  Accept, otherwise reject



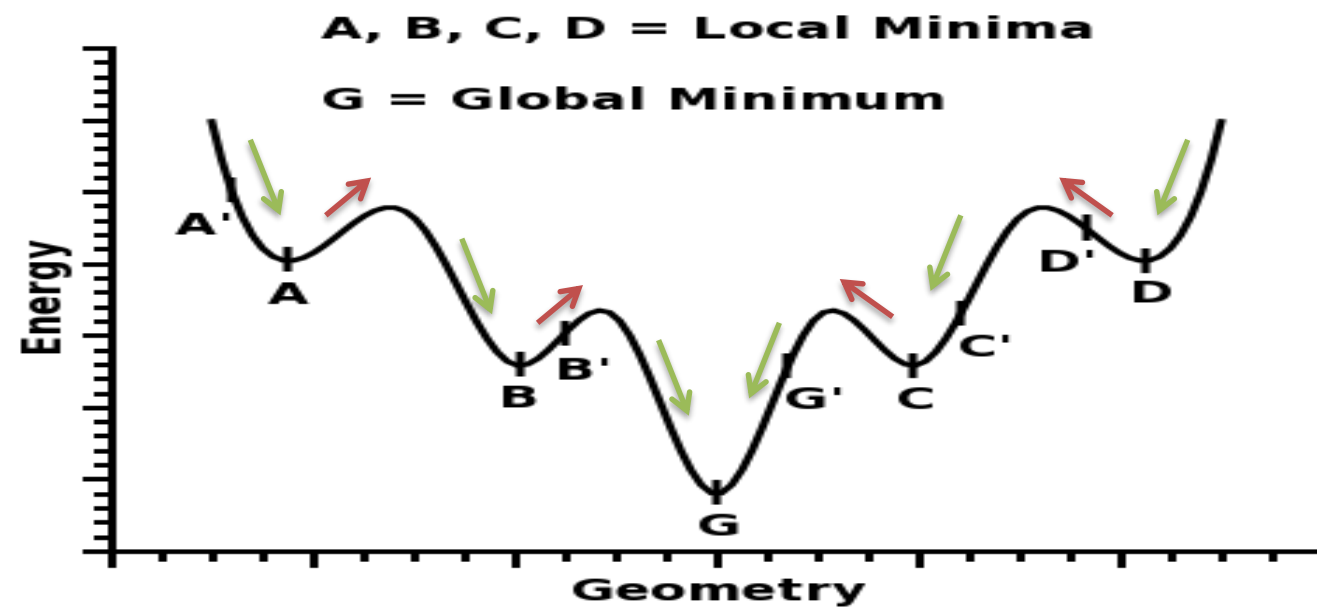
http://en.wikipedia.org/wiki/Simulated_annealing

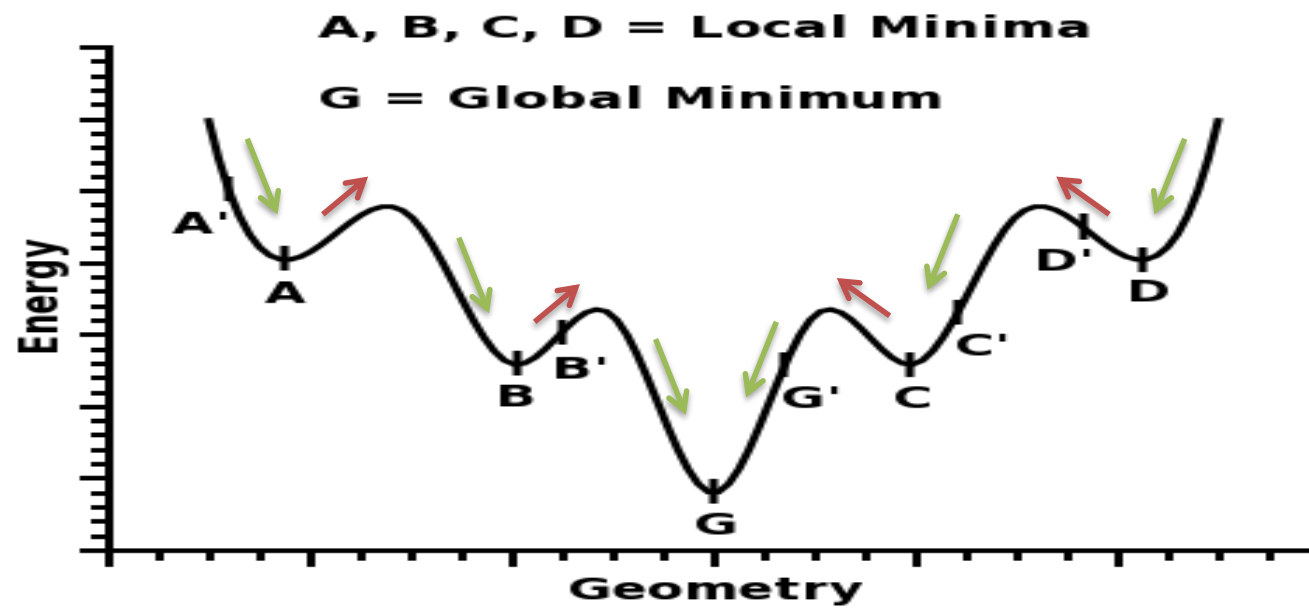
- Origin : Statistical Physics
- Maxwell-Boltzmann : Kinetic Theory of Gases
- Only Thermal Equilibrium at a temperature T could be observed

- n_i : number of particles with kinetic energy E_i

$$\frac{n_1}{n_2} = e^{-\frac{E_1 - E_2}{kT}}$$

- k : Boltzmann constant
- state $\rightarrow x_i$, its energy (cost) $\rightarrow E_i$
- Probability of observing this state : $p(x_i) \approx e^{-\frac{E_i}{kT}}$

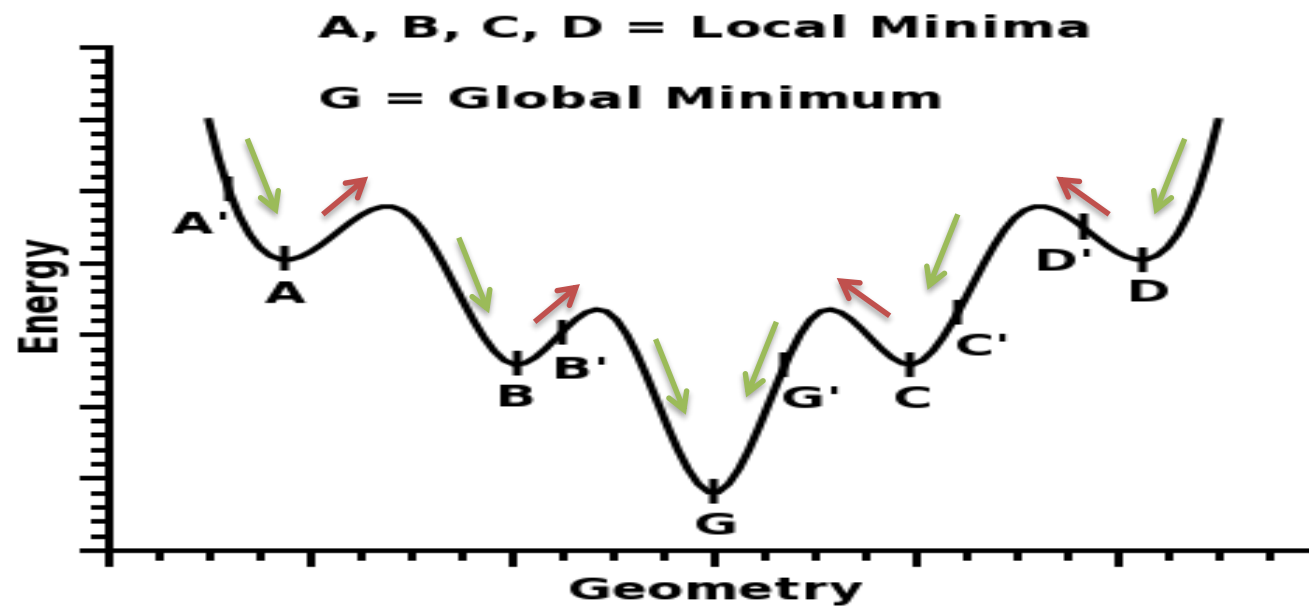




$\longrightarrow E(x(i) - E(x(i - 1))) < 0 \Rightarrow p > 1$

GOING DOWNHILL : Allowed

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$



$\longrightarrow$
 $E(x(i) - E(x(i - 1))) < 0 \Rightarrow p > 1$

GOING DOWNHILL : Allowed

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$

$\longrightarrow$
 $E(x(i) - E(x(i - 1))) > 0 \Rightarrow p < 1$

GOING UPHILL : accepted with a probability $< p$

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$

- For large T : $p \approx 1$ **practically a random search**

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$

- For large T : $p \approx 1$ **practically a random search**

- For small T :
 $E(x(i)) - E(x(i-1)) > 0 \Rightarrow p \approx 0$
 $E(x(i)) - E(x(i-1)) < 0 \Rightarrow p > 1$
practically a gradient descent

$$p = e^{-\frac{(E(x(i)) - E(x(i-1)))}{kT}}$$

- For large T : $p \approx 1$ **practically a random search**

- For small T :

$E(x(i)) - E(x(i-1)) > 0$
 $E(x(i)) - E(x(i-1)) < 0$

$\Rightarrow$
 $\Rightarrow$

$p \approx 0$
 $p > 1$

practically a gradient descent

- COOLING is very important : NOT TOO FAST !

- $T \leftarrow \alpha T \quad 0 < \alpha < 1$ Geometrical cooling
- $T \leftarrow e^{-\alpha T} \quad \alpha > 0$ Exponential cooling
- $T \leftarrow \frac{\alpha}{\ln T} \quad \alpha > 0$ Logarithmic cooling
- $T \leftarrow T - \alpha \quad \alpha > 0$ Linear cooling

- **At high T** : anything can be seen (random search)
- **At low T** : states with low energy can be seen (local minima)
- **At T=0** : **Only zero energy state can be seen (global minimum)**

$$p(x_i) \approx e^{-\frac{E_i}{kT}}$$

Metropolis, Nicholas; Rosenbluth, Arianna W.; Rosenbluth, Marshall N.; Teller, Augusta H.; Teller, Edward (1953). "Equation of State Calculations by Fast Computing Machines". *The Journal of Chemical Physics* **21** (6): 1087

https://web.archive.org/web/20170812013959id_/http://bayes.wustl.edu/Manual/Metropolis.pdf

- **At high T** : anything can be seen (random search)
- **At low T** : states with low energy can be seen (local minima)
- **At T=0** : **Only zero energy state can be seen (global minimum)**

$$p(x_i) \approx e^{-\frac{E_i}{kT}}$$

Metropolis, Nicholas; Rosenbluth, Arianna W.; Rosenbluth, Marshall N.; Teller, Augusta H.; Teller, Edward (1953). "Equation of State Calculations by Fast Computing Machines". *The Journal of Chemical Physics* **21** (6): 1087

Kirkpatrick, S.; Gelatt Jr, C. D.; Vecchi, M. P. (1983). "Optimization by Simulated Annealing". *Science* **220** (4598): 671–680

<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=beb21ee4a3721484b5d2c7ad04e6babd8d67af1d>

- **At high T** : anything can be seen (random search)
- **At low T** : states with low energy can be seen (local minima)
- **At T=0** : **Only zero energy state can be seen (global minimum)**

$$p(x_i) \approx e^{-\frac{E_i}{kT}}$$

Metropolis, Nicholas; Rosenbluth, Arianna W.; Rosenbluth, Marshall N.; Teller, Augusta H.; Teller, Edward (1953). "Equation of State Calculations by Fast Computing Machines". *The Journal of Chemical Physics* **21** (6): 1087

Kirkpatrick, S.; Gelatt Jr, C. D.; Vecchi, M. P. (1983). "Optimization by Simulated Annealing". *Science* **220** (4598): 671–680

Granville, V.; Krivanek, M.; Rasson, J.-P. (1994). "Simulated annealing: A proof of convergence". *IEEE Transactions on Pattern Analysis and Machine Intelligence* **16** (6): 652–65